

LLM-as-Jev: LLMs Are Already Jev-Style Decision Models— When and How to Fine-Tune Them

Yinheng Li
yinhengli@microsoft.com

Justin Wagle
justiwag@microsoft.com

Abstract

Jev-style decision models return categorical probability distributions over predefined options without generating free-form text, enabling software systems to act on their outputs directly. In this work, we investigate the extent to which general-purpose LLMs already possess this capability out of the box, and when fine-tuning is actually necessary. We present LLM-as-Jev, an architecture-preserving framework that extracts calibrated decisions directly from next-token probabilities over bracketed numeric identifiers. LLM-as-Jev provides both a training-free inference recipe and a fine-tuning objective that optimizes candidate selection via a tree-factorized listwise loss while anchoring auxiliary predictions to the base model using KL divergence penalties. Evaluating on Qwen3.5-4B and Qwen3-0.6B, we find that modern LLMs are inherently effective decision models: without training, the 4B model matches community Jev-style models built on the same backbone, outperforms letter-logit readouts, supports arbitrary option counts, and natively handles multimodal decisions over images. Fine-tuning provides targeted rather than universal benefits—substantially improving weaker models and specific tasks (such as many-option intent routing), but offering diminishing returns for strong backbones. Crucially, our KL anchors prevent behavioral degradation in conversational text generation, with LoRA delivering the strongest performance on capable models.

1 Introduction

Software systems routinely make decisions over predefined options: routing a support ticket, identifying a user’s intent, or verifying whether evidence satisfies a given rule. While standard chat models can perform these tasks, their generated responses must be parsed into structured values, introducing latency and the risk of malformed outputs. Jev (Almeida, 2026) addresses this by directly

returning a probability distribution across candidate options without free-form text generation, allowing downstream software to consume its predictions natively. Jev aims to be fast and cost-effective while retaining the broad versatility of an LLM: deploying a new task requires only an instruction prompt and a set of candidate options rather than a dedicated classifier. This paradigm has attracted considerable attention; although Jev’s model weights and training methodologies remain proprietary, the public JevBench leaderboard (Benchmark Heaven, 2026) already lists over 60 community Jev-style models.

A central design choice among these community Jev-style models is whether to update model parameters. Many approaches fine-tune the backbone, frequently introducing custom classification heads or LoRA adapters trained on synthetic questions from teacher models; others evaluate frozen LLMs directly. The resulting performances vary substantially. For instance, Plumb (crh225, 2026) fine-tunes Qwen3.5-4B with LoRA over several iterative rounds of teacher-generated queries and hard-negative mining, achieving 89.6% accuracy on public JevBench items. In contrast, Hopper (HopitAI, 2026) and reflex (kshetrjna12, 2026), both similarly adapted from Qwen3.5-4B, yield performance comparable to SemIf (Lee, 2026), which simply extracts option-letter logits from the unmodified frozen model (82.3% and 79.2% vs. 81.0%). This stark contrast raises two fundamental questions: *to what extent do pre-trained LLMs inherently possess decision-making capabilities, and under what conditions does fine-tuning yield practical gains?*

Our approach: leveraging off-the-shelf LLMs.

We present a framework that turns existing causal LLMs into Jev-style decision models without altering their architectures, tokenizers, or vocabularies, while minimizing dependence on task-

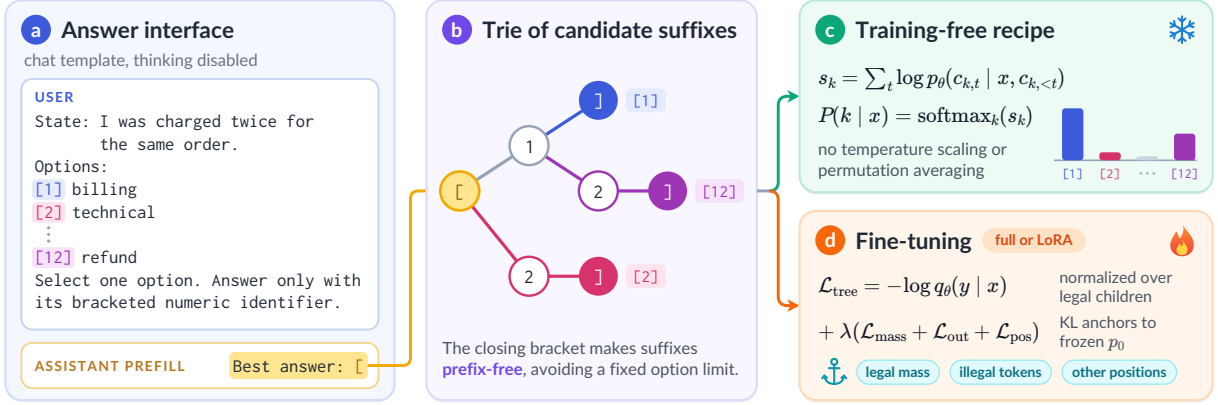


Figure 1: Overview of LLM-as-Jev. (a) Options are numbered, and the answer is prefilled with Best answer: [. (b) The candidate suffixes “ k ” form a prefix tree. (c) The training-free recipe scores each suffix by its log-probability. (d) Fine-tuning trains the choice among candidates with a listwise loss and keeps other predictions close to the original model with KL anchors.

specific training. In contrast to many community Jev-style models that introduce custom classification or pointer heads (Palmer, 2026; mohit67890, 2026; FlyMy.AI, 2026; Cai, 2026; akhilaaa3, 2026; Gribov, 2026) or reserved tokens (Palmer, 2026; FlyMy.AI, 2026)—which must be retrained for every new backbone—our architecture-preserving design provides three distinct benefits:

- **Zero architectural overhead per backbone.** The same prompting convention and parallel scoring procedure apply out of the box to any causal LLM, allowing newly released models to be evaluated immediately and fine-tuned only when targeted adaptation is required.
- **Direct inheritance of base model scaling.** As foundation models advance in general reasoning and factual knowledge, their decision-making accuracy improves correspondingly without necessitating new decision-specific training corpora.
- **Native multimodal support.** Multimodal LLMs equipped with vision encoders can perform image classification and visual question answering through the identical interface without multimodal fine-tuning (§4.2).

Two complementary recipes. LLM-as-Jev introduces two recipes unified under a single interface (Figure 1). The **training-free recipe** formats candidate options with bracketed numeric indices [1]...[K], conditions generation on the assistant prefix Best answer: [, and computes joint probabilities over candidate continuation tokens; it requires no labeled supervision, additional parameters, or vocabulary expansions. The **fine-**

tuning recipe operates through the identical interface, updating either full model weights or LoRA adapters (Hu et al., 2022): a tree-factorized listwise loss trains the model to discriminate among valid candidates, while three auxiliary KL-divergence penalties (*anchors*) prevent probability drift across other generation contexts. Under either configuration, the model remains a fully functional causal LLM capable of standard text generation.

Findings. We evaluate both recipes on Qwen3.5-4B and Qwen3-0.6B across six data compositions alongside ablations over anchor strengths and parameter-efficient tuning. Our evaluations span standard external benchmarks, the public JevBench suite, general linguistic capabilities, multimodal image benchmarks, and conversational behavior. Our findings resolve three core questions:

1. **How capable are LLMs without fine-tuning?** (§4.2) Substantially capable: a pre-trained LLM is inherently an effective decision model. Off-the-shelf Qwen3.5-4B matches competitive community Jev-style models built on the same backbone, outperforms letter-logit extraction, naturally scales to arbitrary candidate counts, and exhibits strong probabilistic calibration. Furthermore, multimodal visual decision-making emerges zero-shot through the same interface.
2. **When does fine-tuning help?** (§4.3) Fine-tuning is primarily beneficial when training data directly covers specific deficiencies of the base model. It yields marked improvements for smaller models and targeted domains (such as high-cardinality intent routing), but provides

diminishing returns—and can even induce negative transfer—when applied indiscriminately to capable models on broad benchmarks. Importantly, this targeted adaptation incurs negligible degradation in general model abilities.

3. **How should one fine-tune?** (§4.4) KL anchors are crucial for preventing degenerate generation outputs and preserving conversational integrity. Stronger models benefit most from LoRA paired with tight anchors ($\lambda = 1$); for smaller models, lighter anchors trade conversational stability for in-domain accuracy, making $\lambda = 1$ the recommended default across scales.

2 Background and Related Work

Jev and JevBench. Jev formalizes decision tasks as categorical distributions over discrete candidate sets across three canonical formats (Almeida, 2026): *choice* (selecting one of K options), *noul* (binary verification, “yes” vs. “no”), and *score* (ordinal grading). JevBench (Benchmark Heaven, 2026) benchmarks such decision models across intelligence, calibration, inference latency, and computational cost. The benchmark provides 231 publicly released items (48 easy, 72 standard, and 111 hard), while held-out and sealed test suites remain private. In this study, we evaluate exclusively on the public items and therefore report diagnostic accuracies rather than official leaderboard standings.

Community Jev-style models. Table 16 (Appendix J) summarizes community Jev-style models closely related to our work. *Frozen-model readouts* extract predictions directly from output token probabilities without updating model parameters. For instance, SemIf (Lee, 2026) and open-alternative-jev (IkerMoel, 2026) read option-letter logits from an unmodified Qwen3.5-4B backbone, whereas Cygnet (blockbrain-ai, 2026) and jqv (Octalab, 2026) apply fitted temperatures to larger language models. AnyJev (Zhang et al., 2026) mitigates option-order and prior biases via cyclic permutations and batch-level calibration. LLM2Jev (Yin-songxu, 2026) also keeps the model frozen but judges each option separately: one prompt per option asks whether that option fits, and the prompts’ yes-versus-no probabilities, normalized across options, form the distribution. LLM-as-Jev instead scores all options jointly in a single prompt (§3.3); without any training, it matches or exceeds all frozen readouts built on Qwen3.5-4B in public JevBench accuracy (§4.2).

Fine-tuned readouts adapt model parameters to improve task-specific alignment. JevK5 (allebee, 2026), Plumb (crh225, 2026), Hopper (HopitAI, 2026), and reflex (kshetrajna12, 2026) train LoRA adapters on Qwen3.5-4B; in particular, JevK5 and Plumb train on letter logits using challenging synthetic queries generated and filtered by teacher models alongside replay of public datasets. Among existing systems, decider (Mapika, 2026) is conceptually closest to our regularization strategy: it performs full-parameter fine-tuning on a base model while encouraging predictions on replayed examples to match the parent model’s answer distribution.

Probabilistic calibration. In automated decision-making systems, predicted probabilities are as critical as the top-1 choices themselves, enabling downstream logic to defer low-confidence cases to human review. Because modern neural networks and pre-trained LLMs frequently suffer from overconfidence (Guo et al., 2017; Desai and Durrett, 2020; Kadavath et al., 2022; Tian et al., 2023), rigorous calibration assessment is essential. We report two standard metrics computed directly from raw model distributions without post-hoc temperature scaling: Expected Calibration Error (ECE) and the Brier score. ECE quantifies the alignment between confidence and empirical accuracy by partitioning predictions into probability bins and computing the bin-weighted absolute discrepancy between average confidence and accuracy. The Brier score measures the mean squared error between the predicted probability vector and the one-hot target, simultaneously rewarding accuracy and probabilistic calibration. For both metrics, lower values indicate superior performance.

3 The LLM-as-Jev Framework

3.1 Problem Formulation

A decision instance consists of a context state s , a set of candidate options $o_1, \dots, o_K$, and a target choice $y \in \{1, \dots, K\}$. The objective is to produce a probability distribution $P(k \mid s)$ over the candidate set. All three Jev-supported task types map cleanly into this formulation: a verification (yes/no) query specifies two options describing the acceptance criteria for “yes” and “no”, while an ordinal scoring query defines an ordered sequence of graded options.

3.2 Answer Interface

We query the model using its default chat template with thinking mode disabled, formatting the user prompt as follows:

```
State: {s}
Options:
[1] {o1} . . . [K] {oK}
Select one option. Answer only with its
bracketed numeric identifier.
```

Each candidate is rendered on a separate line. We condition the assistant’s turn on the fixed prefix `Best answer: [`, such that selecting option k corresponds to generating the completion “ $k]$ ”. We define this completion as the *candidate suffix*, represented by the token sequence $c_k = (c_{k,1}, \dots, c_{k,n_k})$. This design yields three key advantages:

- **Unbounded option capacity.** Because every candidate suffix terminates with a closing bracket, no complete suffix can serve as the prefix of another (e.g., `1]` is not a prefix of `12]`). This prefix-free property supports arbitrarily large candidate sets K , unlike single-letter readouts that are inherently bounded to 16 or 26 options.
- **Shallow suffix trie.** The candidate suffixes naturally form a prefix tree (trie; Figure 1) whose depth is governed by the underlying tokenizer. Tokenizers that process digits individually (such as Qwen’s) yield a trie depth of at most $\lfloor \log_{10} K \rfloor + 2$. Tokenizers that group digits (such as Phi-4’s) encode any integer up to 999 into a single token, producing a trie of depth two. The scoring rule and loss formulation generalize across any such trie structure (Appendix C).
- **Generative compatibility.** The model retains the ability to emit structured answers in standard autoregressive text generation. We utilize this text mode to verify that the model adheres to the expected syntax post-training; fine-tuning leaves general generation capabilities intact (§4.3.4).

3.3 Training-Free Recipe

Let x denote the full input sequence ending with the prefix `Best answer: [`. We score candidate k by accumulating log-probabilities across its suffix tokens, followed by a softmax normalization over

all candidate scores:

$$s_k = \sum_{t=1}^{n_k} \log p_{\theta}(c_{k,t} \mid x, c_{k,<t}), \quad (1)$$

$$P(k \mid x) = \exp(s_k) / \sum_{j=1}^K \exp(s_j).$$

Joint candidate scoring. Because candidate tokens are scored after conditioning on the complete prompt, the model contextualizes each alternative relative to the full candidate slate. In contrast, multiple community Jev-style models score candidates in isolation: leaderboard rerankers and NLI classifiers evaluate each (state, option) pair independently (Benchmark Heaven, 2026), CLM-8B embeds each option separately (Kwok et al., 2026), and jev-local computes individual candidate log-probabilities in separate forward passes (jev-local contributors, 2026). However, many real-world decision tasks inherently require joint comparison—such as identifying the most cost-effective tier, finding the closest semantic match, or determining whether a query falls under “none of the above”. On public JevBench, jev-local achieves only 74.9% accuracy with a frozen Qwen3.5-9B, trailing list-wise readouts on the smaller Qwen3.5-4B (81.4%).

Option ordering and position bias. A known trade-off of joint scoring is susceptibility to option order, reflecting the well-documented position bias in LLMs (Zheng et al., 2024; Pezeshkpour and Hruschka, 2023). We mitigate this during fine-tuning by randomly shuffling candidate order across training instances, ensuring that numeric identifiers carry no static semantic association; consequently, fine-tuned models exhibit greater consistency across cyclic option rotations on MMBench (Appendix F). While inference-time permutation averaging (Zhang et al., 2026) can further reduce residual bias, it incurs proportional computational overhead, and we omit it to prioritize efficiency.

Parallel suffix scoring. Because all candidate suffixes are known *a priori*, they can be evaluated concurrently rather than decoded sequentially. After computing the key-value (KV) activations for the prompt prefix x , we cache them and evaluate candidate branches in parallel via teacher forcing, mirroring the parallel sampling strategy in Jev (Almeida, 2026). Suffixes of identical length can be batched together. With sufficient batch capacity, even a 151-way classification task requires

only one prompt prefill and at most three short forward passes under digit-level tokenizers, or a single forward pass under multi-digit tokenizers.

3.4 Fine-Tuning Recipe

Design objectives. Effective fine-tuning should enhance decision accuracy without collapsing the general-purpose foundation model into a narrow, brittle classifier. We thus aim for minimal behavioral intervention: parameter updates should adjust only the relative preferences among candidate identifiers while preserving all other predictive behaviors of the base model. Specifically, the total probability mass allocated to valid syntax, the internal rankings of non-target tokens, and next-token distributions across unrelated contexts should remain faithful to the original pre-trained model. We achieve this by pairing a tree-factorized listwise loss with Kullback-Leibler (KL) divergence anchors.

Tree-factorized listwise loss. At each node in the candidate suffix trie, only a subset of vocabulary tokens represents valid transitions toward completed suffixes. We designate these tokens as *legal*, and all other vocabulary items as *illegal*. For an internal trie node v , let $\mathcal{L}(v)$ denote the set of legal continuation tokens and $m_\theta(v) = \sum_{w \in \mathcal{L}(v)} p_\theta(w | x, v)$ their aggregate probability mass (*legal mass*). We locally renormalize probabilities over legal continuations at each step along a candidate’s trajectory:

$$q_\theta(k | x) = \prod_{t=1}^{n_k} \frac{p_\theta(c_{k,t} | x, c_{k,<t})}{m_\theta(c_{k,<t})}. \quad (2)$$

The listwise decision loss is defined as the negative log-likelihood of the ground-truth candidate y : $\mathcal{L}_{\text{tree}} = -\log q_\theta(y | x)$. This corresponds to the exact cross-entropy over the tree-factorized categorical distribution across all K candidates. Crucially, evaluating $\mathcal{L}_{\text{tree}}$ requires computing probabilities solely along the *gold path*—the sequence of tokens that spells out target y —making loss computation highly tractable even for high-cardinality tasks ($K = 151$). While q_θ normalizes locally at each branch, it coincides with the unnormalized inference formulation in Eq. (1) whenever internal trie nodes beyond the root assign unit probability mass to legal continuations.

KL anchors. Because $\mathcal{L}_{\text{tree}}$ normalizes locally across valid continuations, it penalizes only relative preferences among legal tokens: the loss

can remain low even if the model drastically suppresses the absolute probability mass assigned to valid formats, leaving illegal continuations and non-decision positions entirely unconstrained. To maintain structural coherence and general conversational competence, we introduce three Kullback-Leibler (KL) divergence anchors against the frozen reference model p_0 (Figure 2):

$$\mathcal{L}_{\text{mass}} = \sum_v \omega_v \text{KL}_B(m_0(v) \| m_\theta(v)), \quad (3)$$

$$\mathcal{L}_{\text{out}} = \sum_v \omega_v \text{KL}(\bar{p}_0(\cdot | v) \| \bar{p}_\theta(\cdot | v)), \quad (4)$$

$$\mathcal{L}_{\text{pos}} = \sum_u \omega_u \text{KL}(p_0(\cdot | u) \| p_\theta(\cdot | u)). \quad (5)$$

Here, KL_B represents the KL divergence between Bernoulli distributions over the partition of legal versus illegal tokens. The distribution $\bar{p}(\cdot | v)$ denotes the probability distribution renormalized over illegal tokens at node v , while u indexes positions outside the candidate suffix. Anchor targets and their respective weighting factors ω are distributed as follows:

- **Trie nodes:** all nodes along the gold path (total weight 0.5) and three randomly sampled off-path nodes (total weight 0.5).
- **Context positions:** the reply start position (weight 0.5) and the position immediately following the closing bracket (weight 0.5).
- **Tail approximation:** we retain the reference model’s top-64 tokens for each anchored distribution (illegal tokens for $\mathcal{L}_{\text{out}}$; full vocabulary for $\mathcal{L}_{\text{pos}}$) and aggregate remaining probability mass into an individual tail bucket. By the data processing inequality, this truncated divergence forms a rigorous lower bound on the full KL penalty while substantially saving computation.

Systematic ablations support each of these choices (Appendix I): omitting context positions or off-path nodes severely inflates auxiliary drift; top-64 truncation captures over 94.5% of each anchored distribution while saving substantial cache memory; and sampling three off-path nodes matches gold-path regularization quality at an acceptable $2.2\times$ computational overhead. The overall training objective combines the listwise loss with the regularizers:

$$\mathcal{L} = \mathcal{L}_{\text{tree}} + \lambda (\mathcal{L}_{\text{mass}} + \mathcal{L}_{\text{out}} + \mathcal{L}_{\text{pos}}), \quad (6)$$

where λ governs regularization strength ($\lambda = 1$ by default; ablations in §4.4).

Training configuration. We support both full-parameter fine-tuning of the language model and LM head, and parameter-efficient tuning via

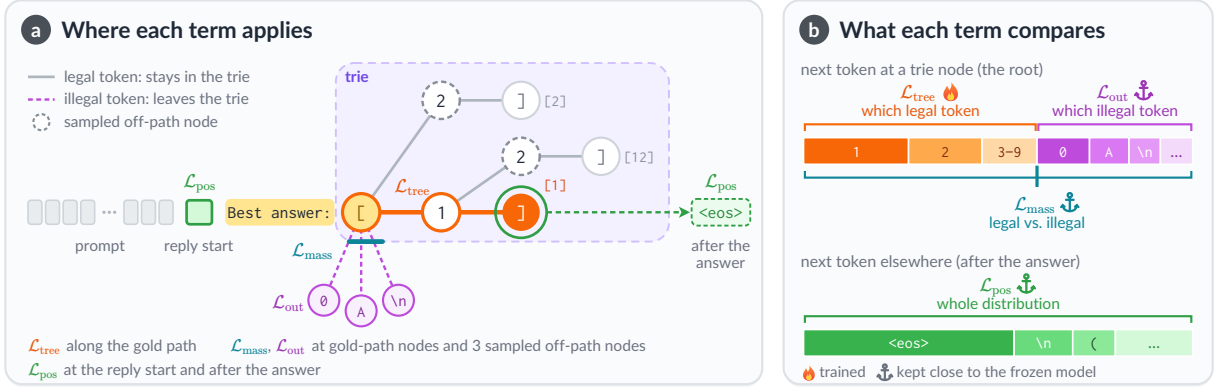


Figure 2: Where the loss terms act, for an example whose correct answer is [1]. (a) $\mathcal{L}_{tree}$ acts along the gold path; $\mathcal{L}_{mass}$ and $\mathcal{L}_{out}$ anchor the total probability and distribution of illegal tokens at trie nodes, and $\mathcal{L}_{pos}$ anchors the next-token distribution at other positions. (b) At a node, $\mathcal{L}_{tree}$ trains only the choice among legal tokens, and the anchors keep the rest close to the original model p_0 .

LoRA (Hu et al., 2022) applied to all linear projections within decoder blocks; LoRA weights are merged into base checkpoints post-training to guarantee identical inference latency. Optimization uses AdamW (Loshchilov and Hutter, 2019) under Fully Sharded Data Parallelism (FSDP; Zhao et al. 2023) with a conservative learning rate, linear warmup and cosine decay, shuffled candidate ordering, and a single training epoch. An automated validation check halts training if the probability mass on legal tokens drops substantially relative to the base model. Complete hyperparameter specifications are detailed in Table 3 (Appendix A).

3.5 Training Data Construction

We curate training sets exclusively from publicly accessible, human-annotated datasets, converting each source into our unified decision template without altering ground-truth labels or soliciting synthetic labels from teacher models. To prevent train-test contamination, we filter all training instances that share a 13-word n-gram (or an identical sentence of six or more words) with any evaluation benchmark, and purge exact duplicates of validation inputs.

Training mixtures were developed iteratively: we began with canonical intent-classification tasks highlighted in early Jev demonstrations (Almeida, 2026), subsequently incorporating reasoning and long-context corpora to expand coverage. Because subsequent data iterations were motivated by error analysis on public JevBench samples, we treat the evaluations in §4.3.3 as exploratory; JevBench was strictly excluded from hyperparameter tuning and checkpoint selection. Section 4.3 details each

Config	Data	Examples	Steps
Intent-100	intent set	3,000	100
Intent-1ep	intent set	95,461	2,984
Reason-9k	R6 + I3 (1k each)	9,000	282
Reason-21k	R6 (3k) + I3 (1k)	21,000	657
Reason-30k	Reason-21k + 3 reasoning sets (3k)	30,000	938
Reason+Long-12k	Reason-9k + 3 long-input sets	12,000	375

Table 1: Training configurations grouped by data domain. R6 and I3 denote reasoning and intent datasets (§4.3; details in Appendix D). Intent-100 is evaluated only on 4B; all other configurations apply to both scales.

data mixture, and Appendix D provides complete dataset breakdowns.

4 Experiments

We outline the experimental setup in §4.1, followed by our core empirical findings: demonstrating that frozen LLMs are effective decision models out of the box (§4.2), delineating the conditions under which fine-tuning provides tangible benefits (§4.3), and establishing optimal fine-tuning and regularization practices (§4.4).

4.1 Setup

Foundation models. We conduct experiments on Qwen3.5-4B (Qwen Team, 2026) (with thinking mode disabled) and the compact Qwen3-0.6B (Yang et al., 2025).

Training configurations. Table 1 summarizes six training configurations spanning varying data mixtures and step budgets (§4.3), all trained under the unified objective in §3.4. In §4.4, we retrain Reason+Long-12k across different anchor weights under both full-parameter tuning and LoRA.

Evaluation dimensions. We evaluate models across five complementary dimensions (sample sizes and evaluation settings detailed in Appendix B):

- **External benchmarks:** seven established classification and reasoning datasets retaining their full candidate sets: BoolQ (Clark et al., 2019), MMLU (Hendrycks et al., 2021), MMLU-Pro (Wang et al., 2024), ARC-Challenge (Clark et al., 2018), WinoGrande (Sakaguchi et al., 2020), SciQ (Welbl et al., 2017), and Banking77 (Casanueva et al., 2020). We report the macro-average accuracy across the first six tasks and isolate Banking77 as a 77-way intent benchmark.
- **JevBench:** the 231 public items, treated strictly as an out-of-distribution diagnostic probe.
- **General capabilities:** general language modeling and task completion assessed via GSM8K (Cobbe et al., 2021), IFEval (Zhou et al., 2023), TriviaQA (Joshi et al., 2017), LAMBADA (Paperno et al., 2016), and WikiText-2 (Merity et al., 2017).
- **Multimodal perception:** visual decision-making evaluated on MMBench (Liu et al., 2024) and MMStar (Chen et al., 2024), testing modalities unseen during fine-tuning.
- **Conversational behavior:** free-form response characteristics and generation stability probed on prompts from Dolly-15k (Conover et al., 2023) and MT-Bench (Zheng et al., 2023).

Metrics and significance testing. We evaluate accuracy, ECE, and the Brier score (§2). Model comparisons are validated using two-sided exact sign tests over the subset of instances on which predictions diverge.

Comparison baselines. We benchmark against four open-source community Jev-style systems using their authors’ official implementations: SemIf, reflex 4B, open-alternative-jev, and Winnow-12B. Across public JevBench items, our local reruns reproduce official leaderboard results within four items. For broader context, we also include proprietary GPT-5.6 Sol evaluated in standard text generation mode.

4.2 An LLM Is Already a Decision Model

Strong zero-shot decision capability at 4B. Without task-specific adaptation, frozen Qwen3.5-4B correctly resolves 188 of the 231 public

System	JevBench			External	
	Acc.	Hard	ECE	Macro	B77
GPT-5.6 Sol (generation)	94.4	88.3	–	91.5	86.0
Winnow-12B [†]	86.6	74.8	.058	82.2	–
4B training-free (ours)	81.4	64.9	.057	78.6	69.0
4B fine-tuned (ours)[†]	80.1	61.3	.070	78.9	75.2
4B LoRA (ours)[†]	84.0	68.5	.050	78.9	75.4
4B generation	81.4	64.9	–	78.4	68.5
SemIf	80.5	60.4	.061	75.4	–
reflex 4B [†]	79.7	59.5	.070	78.8	–
open-alternative-jev	75.8	59.5	.108	75.5	–
0.6B training-free (ours)	56.7	33.3	.278	57.9	22.2
0.6B fine-tuned (ours)[†]	58.0	37.8	.118	59.1	60.5
0.6B LoRA (ours)[†]	61.0	39.6	.121	58.5	59.0
0.6B generation	56.7	33.3	–	57.8	21.9

Table 2: Accuracy comparison across JevBench and External benchmarks (%). [†] Fine-tuned models (all use Reason+Long-12k with $\lambda = 1$, except 0.6B LoRA at $\lambda = 0.01$). Letter readouts cannot evaluate Banking77 ($K = 77$). Full runs in Tables 8 and 12.

JevBench instances (81.4%; Table 2). This performance matches or exceeds dedicated community Jev-style models built on the same backbone, including SemIf (186 correct via single-letter logits; 15 wins vs. 13 losses, $p = 0.85$) and fine-tuned reflex 4B (184 correct). Among evaluated baselines, only the substantially larger Winnow-12B (86.6%) and proprietary GPT-5.6 Sol (94.4%) achieve higher accuracy. Across External benchmarks, our training-free recipe surpasses SemIf by 3.2 percentage points in macro accuracy, winning 313 instances while losing 154 ($p < 10^{-12}$), driven by substantial advantages on MMLU (+123/−56), MMLU-Pro (+96/−41), and ARC-C (+28/−8). Because both systems share identical base weights, these gains confirm the efficacy of joint candidate conditioning and prefix-free scoring.

High-cardinality candidate support. Our framework naturally accommodates all 77 candidate intents in Banking77 within a single inference pass, achieving 69.0% accuracy. In contrast, standard letter-logit readouts cannot be applied to such tasks without fundamentally altering their prompt formulation (§3.2).

Calibrated probabilities without post-hoc scaling. The training-free 4B model achieves an ECE of 0.057 on JevBench out of the box, outperforming both SemIf (0.061) and reflex 4B (0.070, even when incorporating its official calibration mapping).

Consistency between scoring and autoregressive generation. When prompted to generate responses autoregressively under greedy decoding, the 4B model achieves identical accuracy (81.4%) with zero parse failures, selecting the exact same candidate as parallel scoring across all 231 instances. This verifies that our prompt template aligns seamlessly with the model’s natural generation behavior.

Multimodal visual decisions emerge zero-shot. Because LLM-as-Jev introduces no specialized classification heads or modality-specific architectures, any vision-language backbone can evaluate multimodal queries through the identical interface. By prepending image tokens to the prompt, Qwen3.5-4B achieves 82.4% on MMBench under CircularEval (which requires consistent predictions across all option permutations) and 64.6% on MM-Star without any multimodal tuning. Withholding visual inputs collapses performance to 15.2% and 28.8%, confirming that decisions depend authentically on visual comprehension. Crucially, text-only fine-tuning preserves this multimodal competence (§4.3.4).

Format sensitivity in smaller models. In contrast to the 4B backbone, the compact Qwen3-0.6B struggles with zero-shot decision extraction. While its 56.7% JevBench accuracy exceeds the raw-logit leaderboard baseline (48.1%), its probabilistic calibration is poor (ECE 0.278), and performance deteriorates sharply on high-cardinality tasks (22.2% on Banking77). On the 151-class CLINC150 validation set, its initial cross-entropy loss reaches 6.9 nats, performing worse than uniform random guessing ($\ln 151 \approx 5.0$). In autoregressive text mode, the model redundantly emits descriptive option text after the identifier across 219 of 231 queries, yet consistently predicts the candidate selected during scoring. These structural shortcomings highlight where parameter adaptation is genuinely required.

4.3 When Fine-Tuning Helps

Fine-tuning yields meaningful dividends when three conditions converge: the base model exhibits deficient zero-shot accuracy on the target domain, task-aligned supervised data is accessible, and the training distribution faithfully reflects the answer topologies and input length distributions encountered at test time. Compact models and focused enterprise domains (such as high-cardinality intent routing) readily satisfy these criteria. Conversely,

deploying a capable foundation model over heterogeneous decision tasks (as probed by JevBench) fails to benefit consistently, as public data covers only a subset of the necessary competencies. Below, we systematically substantiate these findings: performance gains closely track the coverage of training tasks (§4.3.1), unrepresented formats risk behavioral regression (§§4.3.2–4.3.3), and backbone capacity governs the net utility of adaptation (§4.3.4). Figure 3 summarizes outcomes across all six training configurations, with full metrics reported in Table 8 (Appendix F).

4.3.1 Performance Gains Track Supervised Task Coverage

Empirical improvements concentrate squarely on tasks directly represented in the training mixtures where base models exhibit baseline weaknesses, while unrepresented domains remain largely static (Figure 3). Because every training mixture incorporates intent classification, Banking77—initially among the lowest-scoring External tasks across both scales—improves consistently under all configurations. Similarly, datasets containing common-sense questions boost WinoGrande accuracy at the 4B scale. In contrast, knowledge-intensive benchmarks absent from the training corpora, such as MMLU, MMLU-Pro, and TriviaQA, demonstrate negligible gains. Notably, semantic task structure outweighs superficial vocabulary overlap: MMLU and MMLU-Pro show no advancement despite having the highest lexical similarity to ReClor and LogiQA (Appendix E).

4.3.2 Intent Supervision: Domain Gains Coupled with Heuristic Shortcuts

We initially trained on an *intent set* combining three multi-class intent datasets (Larson et al., 2019; FitzGerald et al., 2023; Bitext, 2024) with CommonsenseQA (Talmor et al., 2019) and HellaSwag (Zellers et al., 2019). While this mixture sharply improves in-domain intent routing on Banking77, it induces detrimental heuristic shortcuts across diverse decision tasks. After a single epoch, error analysis on JevBench reveals four distinct failure modes in the 4B model: an aggressive confirmation bias toward “yes” on binary verification queries, severe suppression of “none of the above” fallback options, reliance on superficial lexical matches, and marked overconfidence. These regressions stem directly from distributional omissions in the training corpus: the intent data lacks veri-

		Steps	Same domain as training data		Different domain from training data			JevBench				Averages	
			Banking77	WinoGrande	MMLU	MMLU-Pro	TriviaQA	Acc.	Long items	"Yes"	ECE	External	General
4B	original	0	69.0	66.2	77.1	47.1	41.3	81.4	21	36	.057	78.6	67.3
	Intent-100	100	+5.6	+1.6	-1.2	-2.9	+0.7	-3.9	+1	+19	+035	-0.3	+0.9
	Intent-1ep	2,984	+7.7	+0.8	-4.0	-3.9	+0.4	-6.5	-4	+13	+077	-1.2	0.0
	Reason-9k	282	+6.9	+2.6	-0.2	-1.6	+0.7	0.0	+2	+8	+010	+0.1	+1.5
	Reason-21k	657	+5.8	+3.4	-0.4	+0.9	-0.6	-2.6	-2	+7	+037	+0.8	+0.5
	Reason-30k	938	+4.9	+1.6	+0.8	-1.0	+0.7	-4.3	-4	+7	+037	+0.2	+1.7
	Reason+Long-12k	375	+6.2	+4.0	+0.2	-1.2	+1.0	-1.3	+1	+9	+013	+0.3	+1.5
0.6B	original	0	22.2	53.6	46.8	22.3	4.0	56.7	7	34	.278	57.9	35.6
	Intent-1ep	2,984	+41.8	-3.4	-1.6	-4.4	-1.0	-3.0	+6	+14	-.074	-1.2	-2.2
	Reason-9k	282	+38.2	+0.2	+1.4	-2.1	0.0	-1.7	+2	+7	-.160	+1.3	+1.2
	Reason-21k	657	+37.9	-3.6	+1.4	-0.9	-1.3	+1.3	+4	+8	-.174	+0.7	0.0
	Reason-30k	938	+37.1	-1.0	+0.9	-1.0	0.0	+2.2	+3	+10	-.190	+1.2	0.0
	Reason+Long-12k	375	+38.3	0.0	+1.1	-1.8	0.0	+1.3	+7	+13	-.160	+1.2	+0.2

change from the original model (color saturates at 10 points) worse better

Figure 3: Change from the original model for each fine-tuning configuration (gray rows: original values). Same-domain tasks match task types in the training data; different-domain tasks test knowledge. Long items: the 37 JevBench items longer than 1,500 tokens; “Yes”: “yes” predictions on the 74 yes/no items. Green is better and red is worse. Table 8 gives absolute values and sign tests.

fication tasks, rarely rewards fallback candidates, can be largely resolved via surface keyword matching, and relies on only four static prompt templates. Crucially, KL anchors cannot prevent such distributional collapse because they permit arbitrary probability redistribution among valid option candidates. Prolonged training exacerbates this degradation: JevBench accuracy drops from 81.4% (zero-shot) to 77.5% at 100 steps and 74.9% after a full epoch, even as in-distribution validation loss continues to decrease monotonically. The affirmative bias emerges as early as step 100, demonstrating that data composition—rather than training duration alone—governs transfer fidelity (Appendix G).

4.3.3 Balancing Verification Formats and Sequence Lengths

To counter the failure modes of intent tuning, we designed targeted reasoning mixtures to test two core hypotheses: whether diversifying task formats cures inductive bias, and how input sequence length impacts transfer (treated as exploratory; details in Appendix G).

Format diversity corrects inductive shortcuts. Replacing narrow commonsense sets with six diverse reasoning benchmarks—spanning balanced rule verification, fallback options (“none of the above”), formal logic, and binary choices (Tafjord et al., 2021; Huang et al., 2019; Yu et al., 2020; Liu et al., 2023; Bisk et al., 2020; Bhagavatula et al., 2020)—effectively eliminates the heuristic shortcuts observed under intent tuning. At 4B, this

balanced mixture (Reason-9k) curtails affirmative confirmation bias, restores well-calibrated probabilities, and matches zero-shot accuracy on JevBench (81.4%), representing our strongest full-parameter 4B model (Figure 3). Thus, diversifying answer formats in training data directly resolves behavioral collapse.

The short-context trap: more data can degrade transfer. Surprisingly, simply scaling data volume without expanding sequence-length diversity proves counterproductive. At 4B, expanding short-context reasoning to 21k and 30k instances (Reason-21k, Reason-30k) systematically degrades JevBench accuracy ($p \leq 0.03$). Crucially, these regressions concentrate predominantly on long-context queries: because the reasoning datasets contain exclusively short prompts (< 570 words), excessive short-context training actively impairs the model’s ability to reason over extended documents.

Length alignment restores long-context reasoning. To resolve this sequence-length mismatch, augmenting the mixture with three long-context corpora (Reason+Long-12k) successfully halts this regression at 4B and doubles long-context accuracy on 0.6B ($p = 0.02$). This highlights an essential design principle: successful decision tuning demands distributional alignment in both task format and input length, as scaling short-context supervision alone actively undermines long-context transfer.

4.3.4 Model Scale Governs Adaptation Utility

Model scale dictates the net return on fine-tuning: compact backbones experience widespread structural gains, whereas capable models benefit strictly along targeted axes. Because baseline Qwen3-0.6B lacks native adherence to the structured answer format, fine-tuning substantially enhances high-cardinality decision accuracy and calibration across all domains. Conversely, because Qwen3.5-4B already exhibits robust format compliance and calibrated decision boundaries out of the box, full-parameter fine-tuning yields no net gain on broad out-of-distribution benchmarks like JevBench (74.9–81.4% vs. 81.4%), with meaningful improvements restricted to targeted domains like Banking77. Only parameter-efficient adaptation via LoRA manages to improve the 4B baseline on JevBench (+2.6 points; §4.4.3). Fine-tuning at both scales preserves general capabilities (with no statistically significant decline in General average; Appendix G), and despite text-only training, the 4B models fully retain zero-shot visual decisions, while actually improving on MMBench (Table 11).

4.4 How to Fine-Tune

Practical deployment entails critical tuning decisions, notably whether to update all parameters or employ parameter-efficient adapters, and how aggressively to anchor auxiliary token distributions. We systematically investigate these design choices by retraining Reason+Long-12k across anchor weights $\lambda \in \{1, 0.1, 0.01, 0\}$ (where $\lambda = 0$ isolates the unconstrained listwise objective) under both full-parameter tuning and LoRA (Figure 4; Appendix H). Our principal recommendations are threefold: retain KL anchors to safeguard generation stability (§4.4.1), maintain default anchor weight at $\lambda = 1$ (§4.4.2), and prioritize LoRA for capable backbones (§4.4.3).

4.4.1 KL Anchoring Safeguards Generative Stability

While varying anchor weights largely preserves aggregate benchmark scores, KL regularization is vital for maintaining behavioral fidelity. Because $\mathcal{L}_{\text{tree}}$ provides no gradient signal prior to the candidate token or following the closing bracket, unconstrained optimization permits auxiliary distributions to drift uncontrollably as an unregularized byproduct of shared weight updates. Standard benchmarks completely conceal this pathology: neither JevBench accuracy nor general lan-

guage modeling metrics decline meaningfully in the absence of anchors. Instead, behavioral collapse manifests in autoregressive generation. Without anchors, the 4B model compulsively continues generating redundant text after the identifier in 69 of the 231 JevBench queries, while the 0.6B model exhibits severe termination failures in conversational chat. Introducing even modest anchor penalties ($\lambda \geq 0.01$) eliminates virtually all format-breaking generations (Appendix H).

4.4.2 Anchor Weight: Gains versus Stability

Lighter anchors trade conversational stability for marginal in-domain accuracy gains, making $\lambda = 1$ the recommended default across both scales (Figure 4). At 4B, $\lambda = 1$ yields peak JevBench accuracy and lowest calibration error under both full-parameter tuning and LoRA. At 0.6B, while relaxing anchors to $\lambda \leq 0.1$ slightly lifts Banking77 intent accuracy, it triggers a statistically significant surge in non-terminating chat replies ($p = 0.007$; Appendix H). Thus, $\lambda = 1$ provides the best balance between task performance and generation integrity.

4.4.3 Parameter-Efficient Adaptation vs. Full Fine-Tuning

We recommend LoRA for capable backbones, whereas on smaller models the two adaptation strategies perform comparably. At 4B, LoRA with $\lambda = 1$ emerges as our strongest model overall, achieving 84.0% on JevBench and the best probabilistic calibration (ECE 0.050), significantly outperforming full-parameter tuning ($p = 0.01$; Table 12). This aligns with prior observations that low-rank adaptation mitigates catastrophic forgetting while constraining unnecessary representation drift (Biderman et al., 2024a). At 0.6B, LoRA closely matches or slightly exceeds full-parameter accuracy across all anchor weights. Crucially, parameter efficiency alone is insufficient to substitute for KL anchors: unregularized LoRA models still suffer syntax termination failures in text mode, and unconstrained LoRA on 0.6B exhibits even more severe conversational drift than full fine-tuning (Appendix H).

5 A Practical Recipe

Synthesizing our empirical results, we propose the following practical guidelines for operationalizing causal LLMs as Jev-style decision systems:



Figure 4: Anchor-weight and LoRA ablations on Reason+Long-12k, from $\lambda = 1$ to no anchors ($\lambda = 0$); dotted lines mark the original model. Scores change little with the anchor weight, whereas behavior degrades as the anchors weaken: text-mode failures are JevBench answers that continue generating past the identifier (4B) or run to the length limit (0.6B), and non-ending chat replies do not end within 1,024 tokens. Filled markers: significant change ($p < 0.05$). Table 12 gives all values.

- **Default to a training-free baseline.** A capable foundation model paired with our numbered prefix-free interface provides a strong, well-calibrated starting point without parameter tuning. For multimodal backbones, visual decision capabilities emerge zero-shot out of the box, and newly published LLMs can be evaluated immediately without training delays.
- **Target adaptation to verified deficiencies.** Fine-tuning should be reserved for scenarios where the pre-trained base model exhibits unambiguous weaknesses (e.g., small parameter footprints or high-cardinality routing tasks) and domain-specific annotated data is available. Practitioners should not expect fine-tuning to universally lift accuracy across diverse, heterogeneous decision tasks.
- **Align training mixtures with target test distributions.** Supervised corpora must comprehensively reflect production characteristics, including binary verification, fallback options (such as “none of the above”), and realistic prompt length distributions. We recommend introducing diverse instruction phrasing and pruning trivial instances that can be resolved via shallow keyword heuristics. Modest budgets of a few hundred gradient steps typically suffice.
- **Enforce KL regularization anchors.** Retaining KL anchors is essential to prevent behavioral drift in auxiliary generation contexts. An anchor weight of $\lambda = 1$ provides a robust default across scales: while lighter anchors slightly boost in-domain intent accuracy in compact

models, they trigger significant conversational drift, and should be used only when in-domain classification strictly supersedes conversational stability.

- **Adopt LoRA for capable backbones.** For medium and large backbones, parameter-efficient adaptation via LoRA attains higher peak accuracy while minimizing unwanted drift from the base model’s representation space.
- **Audit behavior beyond in-domain metrics.** Checkpoint selection should rely on out-of-distribution transfer probes rather than in-distribution training loss, which often continues to drop long after general transfer degrades. Crucially, practitioners should inspect autoregressive completions on open-ended prompts, as standard likelihood metrics fail to detect syntax and generation runaway.

6 Conclusion

We presented LLM-as-Jev, a unified, architecture-preserving framework that extracts calibrated Jev-style decisions from general-purpose causal LLMs through structured prefix-free conditioning, with or without parameter adaptation. Our findings demonstrate that modern LLMs are inherently effective decision models out of the box: without training, Qwen3.5-4B matches competitive community Jev-style models built on the same backbone, supports arbitrary candidate counts, and natively handles multimodal visual decisions. Fine-tuning yields targeted rather than universal improvements, proving most valuable for compact backbones and special-

ized domains while offering diminishing returns on capable foundation models. Finally, our KL divergence anchors prevent generative degeneration across conversational contexts, establishing a reliable, lightweight bridge between generative foundation models and deterministic decision systems.

References

- akhilaaa3. 2026. Jev-Omni. <https://huggingface.co/akhilaaa3/Jev-Omni>.
- allebee. 2026. JevK5. <https://github.com/allebee/jevk5>.
- Diogo Almeida. 2026. Introducing system one models & Jev. TypeSafe AI blog, <https://typesafe.ai/blog/introducing-system-one-models-and-jev>.
- Benchmark Heaven. 2026. JevBench: Jev-class model benchmark (v1.4.2.2). <https://benchmarkheaven.com/jev-models/v1.4.2.2>, accessed 2026-09-28.
- Chandra Bhagavatula, Ronan Le Bras, Chaitanya Malaviya, Keisuke Sakaguchi, Ari Holtzman, Hannah Rashkin, Doug Downey, Scott Wen-tau Yih, and Yejin Choi. 2020. Abductive Commonsense Reasoning. In *International Conference on Learning Representations*.
- Dan Biderman, Jacob Portes, Jose Javier Gonzalez Ortiz, Mansheej Paul, Philip Greengard, Connor Jennings, Daniel King, Sam Havens, Vitaliy Chiley, Jonathan Frankle, Cody Blakeney, and John P. Cunningham. 2024a. LoRA Learns Less and Forgets Less. *Transactions on Machine Learning Research*. ArXiv:2405.09673.
- Stella Biderman et al. 2024b. Lessons from the Trenches on Reproducible Evaluation of Language Models. *arXiv preprint arXiv:2405.14782*.
- Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. PIQA: Reasoning about Physical Commonsense in Natural Language. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Bitext. 2024. Bitext customer support LLM chatbot training dataset. <https://huggingface.co/datasets/bitext/Bitext-customer-support-llm-chatbot-training-dataset>.
- blockbrain-ai. 2026. Cygnet recipe. <https://github.com/blockbrain-ai/cygnet-recipe>.
- Zefan Cai. 2026. Open-Jev. <https://github.com/Zefan-Cai/Open-Jev>.
- Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. 2020. Efficient Intent Detection with Dual Sentence Encoders. In *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*.
- Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Zehui Chen, Haodong Duan, Jiaqi Wang, Yu Qiao, Dahua Lin, and Feng Zhao. 2024. Are We on the Right Way for Evaluating Large Vision-Language Models? In *Advances in Neural Information Processing Systems*.
- Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *arXiv preprint arXiv:1803.05457*.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.
- Mike Conover, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. 2023. Free Dolly: Introducing the World’s First Truly Open Instruction-Tuned LLM. Databricks blog, <https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm>.
- crh225. 2026. Plumb-4B. <https://github.com/crh225/plumb>.
- Shrey Desai and Greg Durrett. 2020. Calibration of Pre-trained Transformers. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*.
- Haodong Duan et al. 2024. VLMEvalKit: An Open-Source Toolkit for Evaluating Large Multi-Modality Models. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 11198–11201.
- EldanRing. 2026. Winnow-12B. <https://huggingface.co/EldanRing/Winnow-12B>.
- Jack FitzGerald et al. 2023. MASSIVE: A 1M-Example Multilingual Natural Language Understanding Dataset with 51 Typologically-Diverse Languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*.
- FlyMy.AI. 2026. Decision fast 0.6b. <https://huggingface.co/flymy-ai/decision-fast-preview>.

- Mikhail Gribov. 2026. typecastlm. <https://github.com/mihail-gribov/typecastlm>.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On Calibration of Modern Neural Networks. In *Proceedings of the 34th International Conference on Machine Learning*.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*.
- HopitAI. 2026. Hopper. <https://huggingface.co/HopitAI/hopper>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.
- Lifu Huang, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2019. Cosmos QA: Machine Reading Comprehension with Contextual Commonsense Reasoning. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*.
- IkerMoel. 2026. open-alternative-jev. <https://github.com/ikermoel/open-alternative-jev>.
- jev-local contributors. 2026. jev-local. <https://github.com/us/jev-local>.
- Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. 2017. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*.
- Saurav Kadavath et al. 2022. Language Models (Mostly) Know What They Know. *arXiv preprint arXiv:2207.05221*.
- Yuta Koreeda and Christopher D. Manning. 2021. ContractNLI: A Dataset for Document-level Natural Language Inference for Contracts. In *Findings of the Association for Computational Linguistics: EMNLP 2021*.
- kshetrajnal2. 2026. reflex. <https://github.com/kshetrajnal2/reflex>.
- Jacky Kwok, Hangoo Kang, Tarun Suresh, Jon Saad-Falcon, Marco Pavone, Christopher Ré, and Azalia Mirhoseini. 2026. Contrastive Language Models: A System One Model for Fast and Generalizable Decision-Making. Notion blog. <https://contrastive-lm.notion.site>. Posted September 23, 2026.
- Stefan Larson, Anish Mahendran, Joseph J. Peper, Christopher Clarke, Andrew Lee, Parker Hill, Jonathan K. Kummerfeld, Kevin Leach, Michael A. Laurenzano, Lingjia Tang, and Jason Mars. 2019. An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*.
- Theodore Lee. 2026. SemIf (formerly OpenJev). <http://github.com/TheoLeeCJ/openjev>.
- Hanmeng Liu, Jian Liu, Leyang Cui, Zhiyang Teng, Nan Duan, Ming Zhou, and Yue Zhang. 2023. LogiQA 2.0—an improved dataset for logical reasoning in natural language understanding. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 31:2947–2962.
- Yuan Liu, Haodong Duan, Yuanhan Zhang, Bo Li, Songyang Zhang, Wangbo Zhao, Yike Yuan, Jiaqi Wang, Conghui He, Ziwei Liu, Kai Chen, and Dahua Lin. 2024. MMBench: Is Your Multi-modal Model an All-around Player? In *European Conference on Computer Vision*.
- Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*.
- Mapika. 2026. decider: One-pass typed decisions with calibrated probabilities. <https://github.com/Mapika/decider>.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer Sentinel Mixture Models. In *International Conference on Learning Representations*.
- mohit67890. 2026. Imajev. <https://github.com/mohit67890/imajev>.
- Octalab. 2026. jqv. <https://github.com/Octalab-Inc/jqv>.
- Jared Palmer. 2026. kev. <https://github.com/jaredpalmer/kev>.
- Richard Yuanzhe Pang, Alicia Parrish, Nitish Joshi, Nikita Nangia, Jason Phang, Angelica Chen, Vishakh Padmakumar, Johnny Ma, Jana Thompson, He He, and Samuel R. Bowman. 2022. QuALITY: Question Answering with Long Input Texts, Yes! In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.
- Denis Paperno, Germán Kruszewski, Angeliki Lazariidou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. 2016. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*.

- Pouya Pezeshkpour and Estevam Hruschka. 2023. Large Language Models Sensitivity to The Order of Options in Multiple-Choice Questions. *arXiv preprint arXiv:2308.11483*.
- Qwen Team. 2026. Qwen3.5-4B model card. <https://huggingface.co/Qwen/Qwen3.5-4B>.
- Anna Rogers, Olga Kovaleva, Matthew Downey, and Anna Rumshisky. 2020. Getting closer to AI complete question answering: A set of prerequisite real tasks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 8722–8731.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2020. WinoGrande: An Adversarial Winograd Schema Challenge at Scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Maarten Sap, Hannah Rashkin, Derek Chen, Ronan LeBras, and Yejin Choi. 2019. SocialQA: Commonsense Reasoning about Social Interactions. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*.
- Tal Schuster, Adam Fisch, and Regina Barzilay. 2021. Get Your Vitamin C! Robust Fact Verification with Contrastive Evidence. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.
- Oyvind Tafjord, Bhavana Dalvi Mishra, and Peter Clark. 2021. ProofWriter: Generating Implications, Proofs, and Abductive Statements over Natural Language. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*.
- Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2019. CommonsenseQA: A Question Answering Challenge Targeting Commonsense Knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.
- Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher D. Manning. 2023. Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multi-hop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics*. ArXiv:2108.00573.
- Yubo Wang et al. 2024. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*.
- Johannes Welbl, Nelson F. Liu, and Matt Gardner. 2017. Crowdsourcing Multiple Choice Science Questions. In *Proceedings of the 3rd Workshop on Noisy User-generated Text*.
- Sean Welleck, Ilia Kulikov, Stephen Roller, Emily Dinan, Kyunghyun Cho, and Jason Weston. 2020. Neural Text Generation With Unlikelihood Training. In *International Conference on Learning Representations*.
- An Yang et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*.
- Yinsongxu. 2026. LLM2Jev. <https://github.com/Yinsongxu/LLM2Jev>.
- Weihao Yu, Zihang Jiang, Yanfei Dong, and Jiashi Feng. 2020. ReClor: A Reading Comprehension Dataset Requiring Logical Reasoning. In *International Conference on Learning Representations*.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a Machine Really Finish Your Sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*.
- Jiamu Zhang, Tianze Yang, Yucheng Shi, and Liang Wu. 2026. AnyJev: Turn any LLM into a Jev-style decision model. <https://github.com/nokia-applied-research/AnyJev>.
- Yanli Zhao et al. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *Proceedings of the VLDB Endowment*. ArXiv:2304.11277.
- Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. 2024. Large Language Models Are Not Robust Multiple Choice Selectors. In *The Twelfth International Conference on Learning Representations*.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*.
- Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Sidhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-Following Evaluation for Large Language Models. *arXiv preprint arXiv:2311.07911*.

A Implementation Details

Prompt. The full user-message template is:

```
State: {state}
Options:
[1] {option 1}
[2] {option 2}
...
Select one option. Answer only with its
bracketed numeric identifier.
```

We apply the model’s chat template with `add_generation_prompt=True` and `enable_thinking=False`.

- **Scoring mode** appends `Best answer: [` to the formatted prompt and computes the log-probability of each candidate suffix “ k ”.
- **Text mode** continues the same prompt, including `Best answer: [`, with greedy generation for up to 32 new tokens and stops at EOS. The concatenated prefix and generated completion must begin with a valid candidate identifier $[k]$, which is extracted as the predicted answer; any trailing text is discarded, including output truncated at the length limit. Generations failing this leading syntax count as parse failures.

We take each candidate’s suffix tokens from the tokenization of the full string, the prompt followed by the suffix, and check that the prompt’s tokens form a prefix of it, so all candidates share one prompt prefix (Appendix C).

Reference statistics. Before training, we use the frozen original model to compute and cache the following anchor statistics for each example:

- the legal mass at each anchored node;
- the top-64 illegal tokens and the mass of the remaining illegal tokens;
- the top-64 tokens and the remaining mass at each anchored position.

Training reads these cached statistics, so it does not need a second model in memory. This top- k truncation substantially compresses cache storage: retaining only the 64 most probable tokens and a single remainder bucket per anchored site requires negligible footprint, compared to storing full vocabulary distributions (151,936 tokens for Qwen3-0.6B; 248,320 for Qwen3.5-4B). Appendix I provides systematic ablations over anchor sites, top-64 truncation, and off-path node sampling.

Latency. Our reference implementation uses FP32 and is not optimized for speed. On a single RTX A5500, median latency is 194 ms for 4B and 76 ms for 0.6B; SemIf takes 46 ms in BF16.

These measurements are indicative, and latency optimization is outside the scope of this work.

Hyperparameters. See Table 3.

Item	Setting
Trainable parameters	language model + LM head (all)
LoRA (§4.4.3)	rank 16, $\alpha = 32$, dropout 0, on decoder linear layers (32.5M trainable parameters for 4B, 10.1M for 0.6B); merged after training
Parallelism, precision	FSDP; BF16 compute, FP32 optimizer states
Optimizer	AdamW; weight decay 0.01; gradient clipping 1.0
Peak learning rate	4B: 2×10^{-6} ; 0.6B: 5×10^{-6} ; LoRA: 10^{-4}
Schedule	linear warmup (10% of steps; 100 for one-epoch runs), linear decay to 10% of the peak; one pass over the data
Global batch size	32 (30 for Intent-100)
Maximum length	8,192 tokens; 10,240 for Reason+Long-12k (inputs never truncated)
Anchors	3 off-path nodes, top-64
Loss weights	listwise 1; anchors $\lambda = 1$ (§4.4: 0.1, 0.01, 0)
Other	shuffled options; gradient checkpointing; seed 20260922
Health check	stop if root legal log-mass drops > 0.5 nats (ablations: warn only)

Table 3: Training hyperparameters.

B Evaluation Details

External. We use fixed samples totaling 7,080 items: BoolQ (500), MMLU (1,200), MMLU-Pro (800), ARC-Challenge (600), WinoGrande (500), SciQ (400), and the full Banking77 test set (3,080). We retain all options, including 10 for MMLU-Pro and 77 for Banking77. The External macro average is the mean accuracy on the first six benchmarks.

JevBench. We use the 231 public items, 37 of which are longer than 1,500 tokens. These items never appear in training, and we do not use them to tune learning rates, select checkpoints, or fit calibration.

General. We run lm-evaluation-harness (Biderman et al., 2024b) on GSM8K (250 items, 5-shot, generated solutions), IFEval (200), TriviaQA (300, 5-shot, closed book), LAMBADA (500), and WikiText-2 (62 documents, perplexity). The General average is the mean accuracy on the first four tasks.

Image. Qwen3.5-4B also reads images, and training leaves its vision encoder unchanged. We place

the image before the state and score the numbered options as usual. MMBench-EN v1.1 dev asks each of its 1,292 questions under every rotation of its options (4,876 prompts) and counts a question as correct only if all rotations are answered correctly (CircularEval). MMStar has 1,500 questions selected so that they require the image; we exclude two whose correct option is empty. We evaluate the original model and the largest configuration in each group of Table 1: Intent-1ep, Reason-30k, and Reason+Long-12k. Qwen3-0.6B has no vision encoder. We use VLMEvalKit’s builds of both benchmarks (Duan et al., 2024), the versions named in Qwen’s model card, and verify them against VLMEvalKit’s published checksums. The state holds a fixed instruction, MMBench’s hint when present, and the question. All models use the original model’s image processor. We down-scale images to at most 1,048,576 pixels so that FP32 inference fits on 24 GB GPUs; this affects 0.5% of MMBench prompts and 2.7% of MMStar prompts. Qwen3.5 assigns image tokens rotary positions from the image grid rather than from their order, so each candidate branch continues from the prompt’s largest position; the cached scores match a full recomputation of every candidate to within 10^{-5} . No 13-word window is shared between the image questions and any training set. Qwen reports 89.4 on MMBench and 78.3 on MMStar for generated answers (Qwen Team, 2026), so our scores are not directly comparable.

Behavior. We check the chat behavior of the ablation models (§4.4). We take 50 prompts from each of Dolly-15k’s eight categories in a fixed seeded order, dropping duplicates and prompts longer than 4,000 characters; any context follows the instruction after a blank line. Classification, closed QA, and information extraction form 150 short-answer prompts, where the right behavior is to answer and stop; the other five categories are long-form. MT-Bench adds 80 two-turn questions, whose second turn follows the model’s own first reply, for 560 replies per model. Decoding uses the chat template with thinking disabled, BF16, greedy search, and no repetition penalty, and stops at the end-of-turn or end-of-text token or after 1,024 new tokens. Each original model and its fine-tuned models run on the same GPU type with the same batch size and order. We count replies that do not end within 1,024 tokens and repetition loops, in which at least half of the word 4-grams are repeats (Welleck et al., 2020).

Some legitimate answers exceed 1,024 tokens (69 of the 4B original’s replies), so we compare each reply with the original model’s reply to the same prompt and count changes in both directions. The loop threshold flags 1 of the 4B original’s replies and 6 of the 0.6B original’s. Two Dolly prompts share a 13-word window with MuSiQue training paragraphs. Dolly-15k is released under CC BY-SA 3.0 and the MT-Bench prompts under Apache 2.0.

Metrics and compute. ECE uses 15 bins. Our models run in FP32 at inference time. Training runs use four A100 80GB GPUs, except Intent-100, which uses three GPUs, a global batch size of 30, and 10 warmup steps. Training Reason-9k takes about 41 minutes for 4B (8.3 seconds per step) and 11 minutes for 0.6B.

Comparison systems. We run reflex 4B, a LoRA adapter, with its authors’ calibration file, and Winnov-12B with 8-bit quantization (Q8).

C Tokenizer Differences

The LLM-as-Jev interface imposes only one formal invariant: the token sequence of the prompt prefix must strictly form a prefix of the full tokenization for each completed candidate. The structural topology of the candidate trie, however, depends on how the underlying tokenizer segments numeric strings. We systematically evaluated three widely used tokenizer families across a 1,000-option prompt (Figure 5); all three yield structurally prefix-free tries for identifiers 1 through 1,000.

Digit grouping. Qwen and Llama 2 tokenize digits individually, so an identifier with d digits takes $d + 1$ tokens, and the trie has depth $\lfloor \log_{10} K \rfloor + 2$. Tokenizers that group up to three digits, such as those of Phi-4 and Phi-4-mini, make every identifier up to 999 a single token followed by the closing bracket, so the trie has depth two. Scoring then needs one short forward step after the prompt pre-fill instead of up to three, and the tree-factorized loss reduces to a flat cross-entropy over the legal first tokens. Beyond 999, identifiers span several tokens again (1000 becomes `1000`), but the closing bracket keeps the suffixes prefix-free, so the number of options remains unlimited.

Word-start markers. SentencePiece tokenizers such as that of Llama 2 treat the start of a string as the start of a word. Tokenized on its own, the

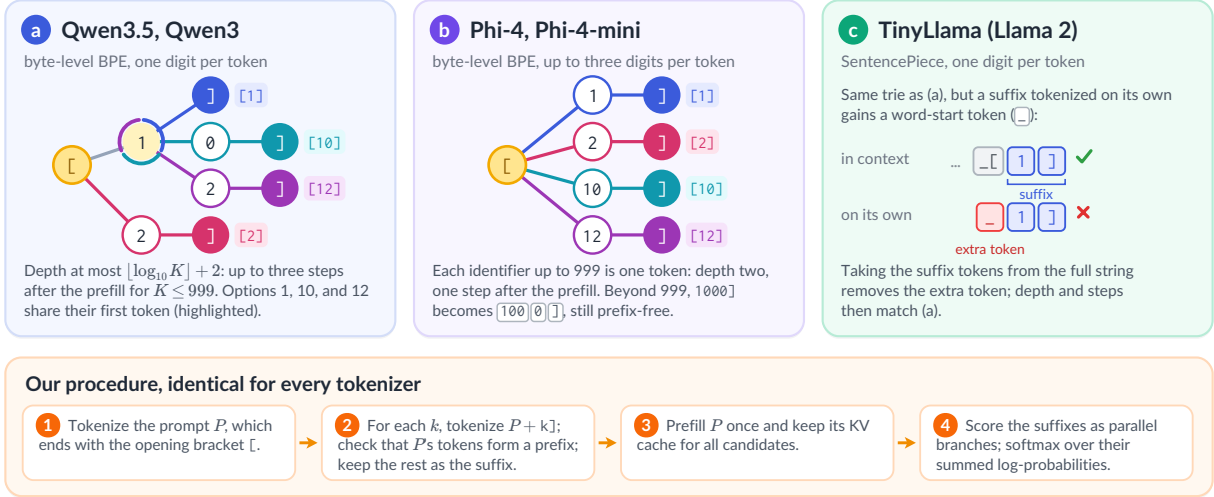


Figure 5: How three tokenizer families split the candidate suffixes, and the procedure that handles all of them. (a) Digit-level tokenizers give a deeper trie in which options 1 and 10 share their first token. (b) Tokenizers that group up to three digits give a depth-two trie. (c) SentencePiece adds a word-start token when a suffix is tokenized on its own; taking suffix tokens from the full string (step 2) avoids it. We checked identifiers 1 to 1,000 with five tokenizers.

suffix $1]$ therefore gains a word-start token that it never has in context, where it follows the opening bracket. We take each suffix’s tokens from the tokenization of the full string instead and check that the prompt’s tokens form a prefix of it. The prompt tokens are then identical across candidates, so parallel decoding with one shared prompt prefill is unaffected. A tokenizer that merged the prompt’s final token with the start of a suffix would fail this check; ending the prefill with an opening bracket avoids such merges for all tokenizers we checked. For Qwen, taking suffix tokens from the full string or from the suffix alone gives identical results.

Isolating the option-10 effect. With tokenizers that split digits individually, the suffixes of options 1 and 10 share their first token, and the next node chooses between the closing bracket and $\emptyset$. Tokenizers that group digits remove this shared node, so they offer a way to isolate the option-10 effect described in Appendix H.

D Datasets

Table 4 lists the training datasets, their licenses, and example counts for each configuration, and Table 5 lists the licenses of the evaluation datasets. In addition to the overlap checks in §3.5, we remove training examples whose inputs also appear in an official validation split.

E Similarity Between Training and Evaluation Data

Data contamination checks. We enforce strict separation between training corpora and evaluation suites by systematically checking for verbatim overlap:

- **Reasoning data vs. External and JevBench.** No 13-word window is shared between the 9,000 training examples and the 7,311 External and JevBench items. Before sampling, the full training sets contained one shared passage: an argument in LogiQA 2.0 also appears in an MMLU item and its MMLU-Pro counterpart. Our overlap filter removed it from the sample.
- **Intent data.** The intent datasets share some label names with Banking77 and JevBench intent items (e.g., `cancel_order`), but no user utterances.
- **General suite.** The same check finds no overlap for the intent set, Reason-9k, Reason-21k, or Reason-30k. Reason+Long-12k has two incidental overlaps from MuSiQue’s Wikipedia paragraphs: one with an IFEval prompt, where evaluation concerns only the output format, and one with a WikiText article that shares a paragraph with a MuSiQue example.

Lexical similarity. After removing instruction templates, we compute TF-IDF vectors over content words. We weight each dataset by its share of the training mixture and compute cosine similarity

Dataset	Task	Options	Intent	Reason-9k	Reason-21k	Reason-30k	Reason+Long-12k	License
CLINC150	intent (10 domains)	151	15,207	1k	1k	1k	1k	CC BY 3.0
MASSIVE (en-US)	intent (voice assistant)	60	11,390	1k	1k	1k	1k	CC BY 4.0
Bitext	intent (e-commerce support)	27	21,839	1k	1k	1k	1k	CDLA-Sharing-1.0
CommonsenseQA	commonsense QA	5	9,139	–	–	–	–	MIT
HellaSwag	situation completion	4	37,886	–	–	–	–	MIT
ReClor	argument reasoning	4	–	1k	3k	3k	1k	research only
LogiQA 2.0	logical reasoning	4	–	1k	3k	3k	1k	CC BY-NC-SA 4.0
CosmosQA	commonsense reading	4	–	1k	3k	3k	1k	CC BY 4.0
PIQA	physical commonsense	2	–	1k	3k	3k	1k	AFL-3.0
α NLI	abductive reasoning	2	–	1k	3k	3k	1k	not stated
ProofWriter	rule verification	3	–	1k	3k	3k	1k	not stated
VitaminC	fact verification	3	–	–	–	3k	–	CC BY-SA 3.0
QuAIL	reading comprehension	4	–	–	–	3k	–	CC BY-NC-SA 4.0
Social IQa	social commonsense	3	–	–	–	3k	–	CC BY 4.0
ContractNLI	contract inference (long)	3	–	–	–	–	1k	CC BY 4.0
MuSiQue	multi-hop answerability (long)	2	–	–	–	–	1k	CC BY 4.0
QuALITY	long-document reading	4	–	–	–	–	1k	CC BY 4.0
Total			95,461	9,000	21,000	30,000	12,000	

Table 4: Datasets per training configuration (citations in §4.3). Median input lengths for long-context datasets are 2.1k (ContractNLI), 2.4k (MuSiQue), and 6.8k tokens (QuALITY). ReClor permits non-commercial research use only; license declarations were absent from official α NLI and ProofWriter releases.

Suite	Dataset	License
JevBench	231 public items	MIT
External	BoolQ	CC BY-SA 3.0
	MMLU	MIT
	MMLU-Pro	MIT
	ARC-Challenge	CC BY-SA 4.0
	WinoGrande	CC BY
	SciQ	CC BY-NC 3.0
	Banking77	CC BY 4.0
General	GSM8K	MIT
	IFEval	Apache 2.0
	TriviaQA	Apache 2.0
	LAMBADA	CC BY 4.0
	WikiText-2	CC BY-SA 3.0
Image	MMBench	Apache 2.0
	MMStar	not stated
Behavior	Dolly-15k	CC BY-SA 3.0
	MT-Bench	Apache 2.0

Table 5: Licenses for official evaluation benchmark releases. MMStar does not provide an explicit license statement.

to each evaluation set (Table 7).

- The reasoning set has higher lexical similarity to all External reasoning benchmarks, largely because ReClor and LogiQA resemble MMLU.
- The intent set has higher similarity to Banking77 and JevBench.

Together with the results in §4.3.1, these comparisons suggest that shared task formats explain the observed gains better than shared vocabulary.

	Intent	Reasoning	JevBench
Examples	95,461	9,000	231
Intent classification	51%	33%	10%
Two-option	0%	22%	32%
Verification	0%	11%	32%
Answer “none”/“unknown”	0.3%	6.0%	3.5%

Table 6: Composition of the intent set, the reasoning set (Reason-9k), and the 231 public JevBench items. Verification includes yes/no and true/false/unknown questions.

Training set	B77	BoolQ	MMLU	Pro	ARC	Wino	SciQ	JevB
Intent	.553	.243	.241	.256	.227	.098	.189	.295
Reasoning	.444	.332	.337	.322	.264	.132	.222	.258

Table 7: TF-IDF cosine similarity between evaluation benchmarks and training mixtures. Pro: MMLU-Pro; Wino: WinoGrande; JevB: public JevBench items.

F Full Results

Table 8 gives the fine-tuning results summarized in Figure 3. Tables 9, 10, and 11 report per-task results for the External, General, and Image suites.

Image results. Table 11 gives the results. None of the three fine-tuned models suffers performance degradation on either benchmark, with all three configurations significantly improving MMBench accuracy: Intent-1ep reaches 84.1% (57 wins vs. 35 losses; $p = 0.03$), Reason-30k reaches 85.3% (57 vs. 19; $p < 0.001$), and Reason+Long-12k reaches 84.9% (49 vs. 16; $p < 0.001$). On MMStar, accuracy increases modestly by 0.9–1.7 points (non-significant across all runs). Almost all MMBench

		JevBench (231 public items)							External		General
Model	Config	Steps	Acc.	Won/lost (p)	Hard	Long/37	ECE	“Yes”	Macro	B77	Avg.
4B	original	0	81.4	–	72	21	.057	36	78.6	69.0	67.3
	Intent-100	100	77.5	12/21 (.16)	65	22	.092	55	78.3	74.6	68.2
	Intent-1ep	2,984	74.9	8/23 (.01)	60	17	.134	49	77.4	76.7	67.3
	Reason-9k	282	81.4	14/14 (1.0)	70	23	.067	44	78.7	75.9	68.8
	Reason-21k	657	78.8	11/17 (.34)	64	19	.094	43	79.3	74.8	67.8
	Reason-30k	938	77.1	10/20 (.10)	60	17	.094	43	78.8	74.0	69.0
	Reason+Long-12k	375	80.1	13/16 (.71)	68	22	.070	45	78.9	75.2	68.8
0.6B	original	0	56.7	–	37	7	.278	34	57.9	22.2	35.6
	Intent-1ep	2,984	53.7	22/29 (.40)	41	13	.204	48	56.7	64.0	33.4
	Reason-9k	282	55.0	11/15 (.56)	36	9	.118	41	59.1	60.4	36.8
	Reason-21k	657	58.0	17/14 (.72)	40	11	.104	42	58.6	60.1	35.6
	Reason-30k	938	58.9	17/12 (.46)	41	10	.088	44	59.0	59.4	35.6
	Reason+Long-12k	375	58.0	17/14 (.72)	42	14	.118	47	59.1	60.5	35.8

Table 8: Fine-tuning performance across training configurations (accuracy in %; Hard and Long denote correct counts on JevBench subsets, with Long covering 37 items > 1,500 tokens). Won/lost pairs report changes relative to base models with two-sided sign-test p -values. “Yes” counts affirmative predictions on 74 binary items (35 ground-truth “yes”). General averages four core language benchmarks. In pooled testing, only 4B Reason-30k shows a significant shift on General ($p = 0.03$).

questions that fine-tuning gains are ones the original model answers correctly under some rotations but not others: 48 of 57 for Intent-1ep, 52 of 57 for Reason-30k, and 48 of 49 for Reason+Long-12k. The fine-tuned models also choose the same option under every rotation more often (90.2–91.3% of questions vs. 87.6%), consistent with training on shuffled options. Each benchmark has six categories. Coarse perception on MMBench improves significantly for all three models; of the other 33 model–category pairs, two change significantly, about as many as chance would produce: MMBench relation reasoning rises for both Reason-30k and Reason+Long-12k (8 wins, 1 loss each). Calibration exhibits minor variance: MMStar ECE ranges from 0.053 (Reason-30k) to 0.093 (Intent-1ep) relative to 0.090 for the base model, and MMBench ECE stays between 0.014 and 0.019 (0.014 for the base model).

G Details of the Data Comparisons

This appendix provides extended qualitative and empirical analyses supporting the findings in §§4.3.2–4.3.4.

Intent data curation. To leverage high-quality human supervision, we began with three public intent-classification benchmarks featuring extensive label sets: CLINC150 (Larson et al., 2019) (151 intents, including an explicit out-of-scope category), MASSIVE en-US (FitzGerald et al., 2023) (60 intents), and Bitext customer

support (Bitext, 2024) (27 intents). We added CommonsenseQA (Talmor et al., 2019) and HellaSwag (Zellers et al., 2019) to cover general commonsense reasoning and narrative continuation. Following cleaning and deduplication, this *intent set* comprises 95,461 instances, roughly half of which represent multi-class intent queries.

Does longer training help? We compared 100 steps on the intent set (Intent-100, 3,000 examples) with one full epoch (Intent-1ep, 2,984 steps). For 4B, longer training makes transfer worse: JevBench accuracy falls from 81.4% to 77.5% after 100 steps and to 74.9% after the full epoch (8 wins, 23 losses, $p = 0.01$). The full-epoch run also reduces External macro accuracy by 1.2 points (pooled $p < 0.001$). Yet the in-distribution development loss continues to decrease monotonically (Bitext: 0.65 $\rightarrow$ 0.001), while the development check based on text generation plateaus at about 500 steps.

Systematic error patterns. Qualitative error analysis of individual predictions from Intent-1ep reveals four systematic failure modes:

- **A bias toward “yes”.** The probability of “yes” increases on 58 of the 74 yes/no items. The number of “yes” predictions rises from 36 to 49, although only 35 items have “yes” as the correct answer. Of the 11 new yes/no errors, 10 occur when the correct answer is “no”.
- **Avoiding “none of these”.** The mean probability assigned to catch-all options falls from 0.056

	System / config	BoolQ	MMLU	Pro	ARC-C	Wino.	SciQ	Macro	B77	ECE
Reference	GPT-5.6 Sol (generation)	91.80	91.42	77.62	98.17	90.40	99.50	91.48	86.04	–
	Winnow-12B (Q8)	89.20	79.67	56.75	95.17	72.60	100.00	82.23	–	.080
	reflex 4B (LoRA)	89.00	75.00	52.50	92.00	64.80	99.50	78.80	–	.056
	SemIf	88.00	71.50	40.25	90.00	62.80	99.75	75.38	–	.080
	open-alternative-jev	87.80	71.92	41.50	89.50	63.00	99.50	75.54	–	.096
4B	original (training-free)	88.00	77.08	47.12	93.33	66.20	99.75	78.58	69.03	.058
	original (generation)	88.00	77.08	46.00	93.33	66.20	99.75	78.39	68.54	–
	Intent-100	89.00	75.92	44.25	92.67	67.80	100.00	78.27	74.64	.085
	Intent-1ep	88.60	73.08	43.25	92.17	67.00	100.00	77.35	76.72	.101
	Reason-9k	88.60	76.92	45.50	92.50	68.80	99.75	78.68	75.91	.067
	Reason-21k	88.60	76.67	48.00	93.17	69.60	100.00	79.34	74.84	.089
	Reason-30k	88.40	77.92	46.12	92.67	67.80	100.00	78.82	73.96	.076
	Reason+Long-12k	88.00	77.25	45.88	92.00	70.20	100.00	78.89	75.19	.087
0.6B	original (training-free)	72.60	46.75	22.25	54.17	53.60	97.75	57.85	22.21	.272
	original (generation)	72.60	46.67	22.12	54.17	53.60	97.75	57.82	21.95	–
	Intent-1ep	73.80	45.17	17.88	57.17	50.20	95.75	56.66	63.99	.158
	Reason-9k	75.20	48.17	20.12	59.17	53.80	98.25	59.12	60.42	.082
	Reason-21k	74.60	48.17	21.38	59.17	50.00	98.00	58.55	60.10	.106
	Reason-30k	72.00	47.67	21.25	62.00	52.60	98.75	59.04	59.35	.097
	Reason+Long-12k	74.40	47.83	20.50	60.33	53.60	97.75	59.07	60.49	.100

Table 9: Detailed accuracy on External benchmarks (%). Macro averages the first six tasks; ECE reports 15-bin calibration across supported tasks. Pro: MMLU-Pro; Wino: WinoGrande; B77: Banking77.

	Config	Steps	Avg.	GSM8K	IFEval	TriviaQA	LAMBADA	LAMBADA ppl	WikiText ppl
4B	original	0	67.3	84.0	80.5	41.3	63.2	5.11	11.47
	Intent-100	100	68.2	85.6	82.0	42.0	63.4	5.11	11.44
	Intent-1ep	2,984	67.3	84.0	79.0	41.7	64.6	4.78	11.62
	Reason-9k	282	68.8	86.0	84.0	42.0	63.0	5.06	11.42
	Reason-21k	657	67.8	84.4	82.0	40.7	64.0	5.00	11.43
	Reason-30k	938	69.0*	86.4	82.5	42.0	65.0	5.03	11.47
	Reason+Long-12k	375	68.8	87.2	82.0	42.3	63.6	4.99	11.44
0.6B	original	0	35.6	44.4	55.0	4.0	39.0	28.21	27.52
	Intent-1ep	2,984	33.4	43.2	48.0*	3.0	39.2	24.34	26.85
	Reason-9k	282	36.8	48.4	54.0	4.0	40.8	25.14	26.31
	Reason-21k	657	35.6	44.8	54.5	2.7	40.4	25.80	26.37
	Reason-30k	938	35.6	44.0	54.0	4.0	40.4	27.74	27.01
	Reason+Long-12k	375	35.8	48.4	51.0	4.0	39.8	26.50	26.64

Table 10: General language modeling and reasoning capabilities (accuracy in %; WikiText perplexity). Avg. denotes four-task mean accuracy. * Statistically significant shift from the base model ($p < 0.05$).

to 0.010.

- **Keyword matching.** On hard multiple-choice items, the model more often follows misleading keyword cues in the question.
- **Overconfidence.** ECE rises from 0.057 to 0.134, and mean confidence in incorrect answers rises from 0.60 to 0.73.

Meanwhile, the same runs gain 5.6 and 7.7 points on Banking77: the data teaches what it covers, and the failures come from what it leaves out. The training data offers plausible explanations for these patterns. It contains no yes/no or verification questions, most intent examples can be solved by keyword matching, and all examples use just four fixed instruction templates. “None of these” is correct in

only 250 of the 95,461 examples. Even Intent-100 shows a “yes” bias (55 “yes” answers), pointing to data composition rather than training duration alone. The same intent data produces a similar bias at 0.6B (48 “yes” answers).

Reasoning data. To counter these empirical failure modes, we curate a diverse mixture of six reasoning datasets (R6). ProofWriter (Tafjord et al., 2021) supplies rule-based verification with balanced true, false, and unknown answers, filling a gap in the intent set and countering the “yes” bias. CosmosQA (Huang et al., 2019) adds questions where “none of the above” is correct; we upsample these to 16%. ReClor (Yu et al., 2020) and LogiQA 2.0 (Liu et al., 2023) require reasoning about argu-

Model	MMBench-EN v1.1 dev (1,292 questions)					MMStar (1,498 questions)		
	Acc.	Won/lost	Coarse	Same	ECE	Acc.	Won/lost	ECE
original	82.4	–	81.2	87.6	.014	64.6	–	.090
Intent-1ep	84.1*	57/35	86.5*	90.5	.014	65.5	126/112	.093
Reason-30k	85.3*	57/19	85.6*	91.3	.018	65.9	113/93	.053
Reason+Long-12k	84.9*	49/16	84.5*	90.2	.019	66.3	108/82	.086
original, no image	15.2	–	9.1	30.8	.214	28.8	–	.234
uniform guess	2.2	–	2.5	–	–	26.9	–	–

Table 11: Zero-shot visual decision performance for Qwen3.5-4B (accuracy in %). MMBench reports CircularEval accuracy (requiring correct predictions across all option permutations). Coarse denotes the 362 coarse-perception queries; Same measures prediction invariance across rotations. * Statistically significant shift from the base model ($p < 0.05$).

ments rather than matching keywords. PIQA (Bisk et al., 2020) and α NLI (Bhagavatula et al., 2020) add two-option commonsense questions. Two-option questions account for 32% of JevBench but were absent from the intent set. We retain the three intent-classification datasets (I3) to preserve intent skills and remove CommonsenseQA and HellaSwag. Because the intent-data experiments showed early in-domain gains, we start with only 1,000 examples per dataset and a single pass (Reason-9k, 282 steps). Table 6 compares the two mixtures.

Does harder, more balanced data help? This mixture successfully curbs the affirmative bias, improves calibration relative to intent tuning, and matches zero-shot JevBench accuracy: with Reason-9k, the 4B model achieves 81.4% (14 wins, 14 losses), marking the top result among all full-parameter 4B configurations. Its ECE of 0.067 represents the best calibration among full-parameter 4B runs (though slightly trailing the base model’s 0.057), while “yes” predictions drop to 44. External macro accuracy reaches 78.7% (+0.1 points). Among individual tasks, only Banking77 shifts significantly (+6.9 points, $p < 10^{-20}$), while variations on WinoGrande (+2.6) and MMLU-Pro (−1.6) remain within statistical margins.

Does adding more reasoning data help? We expand the reasoning set in two ways. Reason-21k uses 3,000 examples per reasoning dataset (657 steps). Reason-30k adds three more datasets with 3,000 examples each (938 steps): VitaminC (Schuster et al., 2021) for fact verification, QuAIL (Rogers et al., 2020) for reading comprehension with “not enough information” answers, and Social IQa (Sap et al., 2019) for social commonsense. Neither improves the 4B model’s JevBench accuracy, which

falls to 78.8% and 77.1%, respectively. Against Reason-9k, Reason-21k has 0 wins and 6 losses ($p = 0.03$) and Reason-30k 0 wins and 10 losses ($p = 0.002$). At 0.6B, more reasoning data does not hurt: Reason-30k wins 14 items and loses 5 against Reason-9k ($p = 0.06$). External macro accuracy remains nearly unchanged (78.7–79.3%).

Where does performance decline? The losses concentrate on the 37 long items. Reason-9k answers 23 correctly, compared with 19 for Reason-21k ($p = 0.13$) and 17 for Reason-30k ($p = 0.03$); 6 of the 10 items that Reason-30k loses against Reason-9k are long, although long items make up only 16% of JevBench. Both the intent and reasoning mixtures contain prompts no longer than about 570 words, whereas hard JevBench items include long policy documents and multi-hop questions. This mismatch suggests that more training on short inputs can hurt performance on long ones.

Long-input data. To address the length mismatch, we add three long-input reasoning datasets to Reason-9k, with 1,000 examples each: ContractNLI (Koreeda and Manning, 2021) for entailment over full contracts, MuSiQue (Trivedi et al., 2022) for deciding whether 20 paragraphs contain enough information to answer a multi-hop question, and QuALITY (Pang et al., 2022) for questions about long articles. Their median input lengths range from 2.1k to 6.8k tokens. The resulting configuration, Reason+Long-12k, uses 12,000 examples and 375 steps.

Does long-input data improve the long items? At the 4B scale, incorporating long-context data restores long-sequence accuracy to baseline levels. Specifically, Reason+Long-12k correctly answers 22 long items, comparable to 21 for the base model and 23 for Reason-9k (yielding 6 wins against 1

loss relative to Reason-30k; $p = 0.13$), and attains 80.1% on JevBench (9 wins and 2 losses relative to Reason-30k; $p = 0.065$). While it does not surpass Reason-9k (81.4%; 1 win, 4 losses, $p = 0.38$), which remains our strongest full-parameter 4B configuration, long-context supervision successfully reverses degradation without over-fitting the original model or Reason-9k. Reason+Long-12k also trains for fewer steps than Reason-30k (375 vs. 938), which may contribute to the recovery. At 0.6B, long-context supervision provides decisive gains: Reason+Long-12k resolves 14 long items correctly, doubling the base model’s score (7 wins, 0 losses; $p = 0.02$) and outperforming all other training configurations (9–13).

The 0.6B model gains broadly. Banking77 accuracy rises by 37.1–41.8 points, Reason-9k reduces CLINC150 development loss from 6.91 to 0.91, and ARC-Challenge gains 3.0–7.8 points. Calibration improves as well: JevBench ECE falls from 0.278 to 0.088–0.204, and mean External ECE from 0.272 to 0.082–0.158. JevBench accuracy stays close to the original (53.7–58.9% vs. 56.7%; best: Reason-30k), and the best 0.6B result is LoRA at $\lambda = 0.01$ (61.0%; §4.4.3).

The 4B model gains only where it is weak. No full-parameter configuration beats the original 4B model on JevBench (74.9–81.4% vs. 81.4%; Reason-9k ties it). External macro accuracy stays within -1.2 to $+0.8$ points of the original, while Banking77 gains 4.9–7.7 points and WinoGrande 0.8–4.0.

General ability is preserved. No fine-tuned model shows a significant decline in the General average. The 4B model moves from 67.3% to 67.3–69.0% and the 0.6B model from 35.6% to 33.4–36.8%; the only significant change in an average is a 1.7-point gain for 4B Reason-30k (pooled $p = 0.03$; all others $p \geq 0.08$). WikiText perplexity remains stable (4B: 11.47 $\rightarrow$ 11.42–11.62; 0.6B: 27.52 $\rightarrow$ 26.31–27.01). The only significant per-task change is a 7.0-point IFEval drop for 0.6B Intent-1ep ($p = 0.04$; Table 10).

H Ablation Details

Experimental setup. The ablations in §4.4 reuse the Reason+Long-12k data and schedule (375 steps, 38 warmup steps) and every other setting; only the development health check warns instead of stopping. LoRA adapters wrap every linear layer in-

side the decoder layers of the language model; embeddings, the LM head, and the vision encoder of Qwen3.5-4B stay frozen. Table 12 gives the results summarized in Figure 4, Table 13 gives per-task results, and Table 14 gives the drift, text-mode, and chat-behavior measurements. Because each configuration is trained with a single random seed, minor fluctuations of one to two percentage points on individual benchmarks (such as Banking77) likely reflect standard run-to-run variance rather than systematic algorithmic divergence.

Aggregate benchmark scores are largely insensitive to anchor strength. JevBench accuracy remains largely invariant to regularization strength: relative to $\lambda = 1$, performance fluctuates by at most 1.7 points at 4B and 3.0 points at 0.6B, with the sole outlier being 4B LoRA under $\lambda = 0.01$ (79.2% vs. 84.0%). Without anchors, the listwise loss fits the development data as well as with them (4B tree loss 0.421 vs. 0.410) and does not collapse: in every development dataset, legal tokens keep at least 91% of the probability at the first answer position. General averages do not fall either, and at 4B lighter anchors even raise them (up to $+2.3$ points at $\lambda = 0.01$, $p = 0.02$). An early run that used only the listwise loss did collapse: legal mass fell to about e^{-17} , JevBench accuracy to 52.8%, and none of the 231 text-mode answers could be parsed. That run motivated the anchors, but it also used a constant learning rate of 10^{-5} , no warmup, and a batch size of 3, so we cannot attribute its collapse to the missing anchors alone.

Unanchored models exhibit severe generation runaway. The listwise loss gives no gradient at the start of the reply or after the answer, so predictions there change only as a side effect of shared weights. Without anchors, the KL divergence from the original model at these two positions grows 44- and 35-fold at 4B, and 51- and 16-fold at 0.6B, relative to $\lambda = 1$ (Table 14). In text mode, the 4B model then keeps writing after the identifier on 69 of the 231 JevBench items: 44 answers run to the 32-token limit, mostly with an explanation, and 25 append the option label. When parsing strictly from the leading bracketed identifier, all generated responses agree with the candidate selected during scoring, maintaining an effective text accuracy equal to scoring accuracy (79.7%). These syntax runaways concentrate predominantly on long-context inputs (35 of the 37 long items). For 0.6B, 176 answers reach the limit, compared

with 1 at $\lambda = 1$. A small weight prevents most of this: at $\lambda = 0.01$, only one 4B answer continues after the identifier (two with LoRA), while 26 0.6B answers reach the limit.

Unregularized drift compromises open-ended conversational stability. The behavior check tests whether this drift extends beyond the decision prompt. At 4B, it does not: under every recipe, including $\lambda = 0$, about as many chat replies newly fail to end as newly end (19 vs. 21 without anchors), and loops stay rare. Trained 4B models only answer short-answer prompts more tersely; at $\lambda = 1$, for example, the full-parameter model’s replies are shorter than the original model’s on 103 of the 150 such prompts and longer on 35. At 0.6B, the change grows as the anchor weight falls. With full-parameter training, 7, 13, 12, and 20 replies newly fail to end at $\lambda = 1, 0.1, 0.01$, and 0, against 1–2 in the other direction; every weight below 1 is significant ($p \leq 0.013$). Most come from long-form Dolly prompts and are often endless numbered lists, as in brainstorming answers. The General scores do not reveal this change: the 0.6B model without anchors has a General average 1.0 point higher than the original, because these tasks score likelihoods, extract the final answer, or check only the stated instructions.

Lighter anchors trade behavioral stability for in-domain gains. Across both scales, reducing anchor regularization trades conversational stability for marginal in-domain accuracy, confirming $\lambda = 1$ as the most reliable default (Figure 4). At 0.6B, no anchor weight produces statistically significant shifts on JevBench relative to $\lambda = 1$: full-parameter models achieve 57.1–59.3% and LoRA reaches 58.0–61.0%, with peak accuracy attained by LoRA at $\lambda = 0.01$ (61.0%; 10 wins and 3 losses against LoRA at $\lambda = 1$, $p = 0.09$) and optimal calibration delivered by full-parameter tuning at $\lambda = 1$ (ECE 0.118). While milder regularization lifts Banking77 accuracy under full fine-tuning ($\lambda = 0.01$: 62.4% vs. 60.5%, $p < 0.001$; $\lambda = 0.1$: 62.2%), all settings below $\lambda = 1$ induce statistically significant increases in non-terminating chat replies, whereas $\lambda = 1$ maintains behavioral fidelity closest to the base model (7 newly non-ending vs. 1 resolved, $p = 0.07$). At 4B, the strongest anchor decides best: $\lambda = 1$ gives the highest JevBench accuracy with both methods and, with full-parameter training, higher Banking77 accuracy than $\lambda = 0$ ($p = 0.002$), though not significantly

higher than $\lambda = 0.1$ ($p = 0.06$) or $\lambda = 0.01$. This suggests that the 4B model already decides well and mainly needs the answer format, so departures from the original model add little. Lighter anchors raise full-parameter 4B External accuracy by about 0.7 points ($\lambda = 0.1$: $p = 0.04$; $\lambda = 0$: $p = 0.03$), partly because $\lambda = 1$ moves ten-option MMLU-Pro answers from option 10 to option 1, whose identifiers share their first token (Appendix H).

Superiority of LoRA on capable backbones.

LoRA configured with $\lambda = 1$ achieves 84.0% on JevBench, marking our strongest 4B model overall (13 wins, 7 losses against the training-free baseline), accompanied by the best probabilistic calibration across all 4B configurations (ECE 0.050 vs. 0.057 for the base model) and the top JevBench Brier score (0.240 vs. 0.247). Three of the four LoRA configurations outperform the training-free baseline (82.3–84.0%), with only $\lambda = 0.01$ dropping to 79.2%. Furthermore, LoRA surpasses full-parameter tuning at $\lambda = 1$ (+3.9 points; 10 wins vs. 1 loss; $p = 0.01$), maintains advantages at $\lambda = 0.1$ (+3.0, $p = 0.07$) and $\lambda = 0$ (+2.6, $p = 0.15$), and ties at $\lambda = 0.01$ (6 wins vs. 7 losses); notably, LoRA at $\lambda = 1$ significantly outperforms every individual full-parameter run ($p \leq 0.02$). At this anchor weight, LoRA matches full fine-tuning on both External (78.9%) and Banking77 (75.4% vs. 75.2%). General benchmark averages (66.7% vs. 67.3%), autoregressive text compliance, and conversational behavior exhibit no significant degradation, altering predictions on only 60 of 1,250 General evaluation items.

Near-parity between LoRA and full fine-tuning at small scale.

At $\lambda = 1, 0.1$, and 0, LoRA and full-parameter training are within one JevBench item of each other; at $\lambda = 0.01$, LoRA reaches 61.0% against 57.1% (12 wins, 3 losses; $p = 0.04$), the best 0.6B result, though not significantly above LoRA at $\lambda = 1$ ($p = 0.09$). LoRA trails on Banking77 at $\lambda = 1$ (54.9% vs. 60.5%) and is slightly lower on External at $\lambda \leq 0.01$, and both methods lose some IFEval accuracy (LoRA 2.0–4.0 points, full-parameter training 3.0–5.0). A higher learning rate (3×10^{-4}) fits the training data slightly better than full-parameter training (development tree loss 0.831 vs. 0.838) but does not raise JevBench (58.4% vs. 58.0% at 10^{-4}) or External accuracy (Table 13).

Parameter efficiency does not obviate regularization. Parameter-efficient training might be expected to protect the original model on its own, but it does not. Without anchors, 9 of the 4B LoRA model’s text-mode answers continue after the identifier, mostly with the option label, and 108 of the 0.6B LoRA model’s answers reach the 32-token limit. In chat, 0.6B LoRA drifts more than full-parameter training at $\lambda \leq 0.01$: 19 and 38 replies newly fail to end at $\lambda = 0.01$ and 0, compared with 12 and 20; at $\lambda = 0.1$ the two are similar (11 vs. 13).

Option 1 versus option 10. MMLU-Pro is the only ranked External benchmark with two-digit identifiers: 658 of its 800 questions have ten options, and the suffixes of options 1 and 10 share their first token. Training shifts answers away from option 10, more so at larger anchor weights. With full-parameter training, the 4B model picks option 10 on 45 of these questions before training and on 31, 32, 25, and 24 at $\lambda = 0, 0.01, 0.1$, and 1; the 0.6B model does so on 18 before training and on 16, 9, 7, and 2. At 4B with $\lambda = 1$, the displaced answers go to option 1 (147 picks vs. 123), and the model loses 15 of these questions net, more than its total net MMLU-Pro loss of 10. We have not identified the cause.

I Anchor Sites and Sizes

Setup. To test whether each anchor site is needed, we retrain Reason+Long-12k with $\lambda = 1$ and all parameters at both scales, removing one part of the anchors at a time (Table 15): the two context positions, the off-path nodes (so that only the gold path is anchored), or the two distribution anchors $\mathcal{L}_{\text{out}}$ and $\mathcal{L}_{\text{pos}}$, which leaves the legal mass alone and is equivalent to keeping $k = 0$ tokens. To size the truncation and the number of off-path nodes without training more models, we compare the original model with the trained models on the first 64 development examples of each of the 12 training datasets (the *analysis examples*), at the sites that training anchors; there we sample eight off-path nodes per example instead of three.

Every anchor site serves a distinct regularizing role. While no single site ablation significantly degrades JevBench accuracy ($p \geq 0.38$), each omission directly destabilizes the specific distribution it targets. Omitting context positions inflates KL divergence at the reply start by $9.4\times$ at

4B and $12.6\times$ at 0.6B (and $2.4\times$ and $2.6\times$ after the bracket); consequently, three text-mode responses emit runaway text at 4B and three reach the length limit at 0.6B, while JevBench ECE rises from 0.070 to 0.089 at 4B and from 0.118 to 0.143 at 0.6B. Removing off-path node anchors increases KL divergence over illegal tokens at unvisited branches by $2.9\times$ at 4B (0.084 to 0.241) and $1.8\times$ at 0.6B (0.176 to 0.319), elevating 0.6B ECE to 0.165. Constraining legal mass alone fails to prevent generative runaway at both scales (16 4B completions append trailing text and 39 0.6B completions hit the length cap), causing reply-start KL to surge 95-fold at 4B and 40-fold at 0.6B—comparable to unconstrained training without any anchors (44- and 51-fold). In conversational chat, omitting off-path nodes or isolating legal mass induces significantly more non-terminating replies than the base model at 0.6B. Variations on Banking77 remain bounded within ± 1.8 points; given single-seed runs, we do not attribute these modest shifts to individual anchor components.

Top-64 truncation captures the distribution tail.

At candidate trie nodes, the base model’s 64 most probable illegal tokens account on average for 94.5–98.9% of total illegal probability mass (rising to 97.1–99.6% with 1,024 tokens), while at context positions the top-64 tokens cover over 99.9% of next-token probability. By the chain rule of KL divergence, the true divergence exceeds our truncated, bucketed objective only by the product of the base model’s tail mass and the residual divergence within the tail; the bound is therefore exceptionally tight when tail mass is light. Across anchored models, this truncation gap averages at most 7.3×10^{-4} nats per anchored site (1.9×10^{-3} without anchors). Moreover, caching truncated distributions consumes only ≈ 0.5 KB per distribution, compared to 0.6 MB (Qwen3-0.6B) or 1.0 MB (Qwen3.5-4B) for the full vocabulary in FP32. Truncating aggressively to $k = 0$ completely removes distributional constraints, reproducing the degenerate behavior observed in legal-mass-only runs.

Sampling three off-path nodes is computationally optimal.

Because a four-option query contains exactly three off-path candidate nodes, setting the sample size to three provides exhaustive off-path coverage for 9 of the 12 training datasets in Reason+Long-12k; only the three intent datasets (spanning 25–150 options) require stochastic subsampling across iterations. Regularizing three ran-

Model	Method	λ	JevBench		External		General	Behavior	
			Acc.	ECE	Macro	B77	Avg.	Text	Chat
4B	original	–	81.4	.057	78.6	69.0	67.3	0	–
	Full	1	80.1	.070	78.9	75.2	68.8	0	20/20
	Full	0.1	79.7	.086	79.6	74.4	69.0	0	13/20
	Full	0.01	79.7	.091	79.0	74.9	69.6*	1	16/17
	Full	0	79.7	.084	79.6	74.2	69.1*	69	19/21
	LoRA	1	84.0	.050	78.9	75.4	66.7	0	15/23
	LoRA	0.1	82.7	.059	79.6	74.7	67.7	0	16/22
	LoRA	0.01	79.2	.081	79.8	74.1	68.6	2	12/22
	LoRA	0	82.3	.070	79.5	73.6	68.6	9	16/23
0.6B	original	–	56.7	.278	57.9	22.2	35.6	0	–
	Full	1	58.0	.118	59.1	60.5	35.8	1	7/1
	Full	0.1	58.4	.121	59.2	62.2	36.1	6	13/2*
	Full	0.01	57.1	.144	59.0	62.4	36.6	26	12/2*
	Full	0	59.3	.141	58.6	61.6	36.6	176	20/2*
	LoRA	1	58.0	.159	58.3	54.9	34.9	1	7/1
	LoRA	0.1	58.9	.150	58.9	60.2	34.6	1	11/2*
	LoRA	0.01	61.0	.121	58.5	59.0	35.5	4	19/2*
	LoRA	0	59.7	.165	57.8	59.8	35.1	108	38/2*

Table 12: Anchor-weight and LoRA ablations on Reason+Long-12k (accuracy in %). λ weights KL anchor penalties ($\lambda = 0$ denotes unconstrained listwise loss). Text reports JevBench text-mode syntax failures (continuations past identifier for 4B; 32-token limits for 0.6B). Chat denotes newly non-terminating vs. newly terminating responses across 560 conversational queries. * Statistically significant shift from the base model ($p < 0.05$). Table 13 gives per-task results.

domly sampled nodes per training instance maintains all off-path branches close to the base model: on held-out analysis instances where eight off-path nodes are evaluated per intent query, off-path KL divergence (0.084 at 4B, 0.176 at 0.6B) remains virtually identical to gold-path divergence (0.089 and 0.192), where every node is anchored deterministically. Increasing the number of sampled nodes yields diminishing returns while imposing substantial computational overhead, as each anchored node requires an additional forward evaluation over the prompt prefix. Sampling three nodes increases evaluated forward tokens 3.4-fold per instance, resulting in a $2.25\times$ slowdown at 4B and $2.23\times$ at 0.6B relative to unanchored training.

J Community Jev-Style Models

Table 16 summarizes the community Jev-style models most closely related to our work on JevBench v1.4.2.2. Information comes from each system’s README or model card and the evaluator’s notes, accessed on 2026-09-28 (LLM2Jev: 2026-10-04). “Public” reports official accuracy on the 231 public items. “Sealed” reports accuracy on 308 unreleased items, where random guessing achieves about 29.3%.

Model	Method	λ	External								General					
			BoolQ	MMLU	Pro	ARC-C	Wino.	SciQ	Macro	B77	GSM8K	IFEval	TriviaQA	LAMBADA	Avg.	WikiText ppl
4B	original	–	88.0	77.1	47.1	93.3	66.2	99.8	78.6	69.0	84.0	80.5	41.3	63.2	67.3	11.47
	Full	1	88.0	77.2	45.9	92.0	70.2	100.0	78.9	75.2	87.2	82.0	42.3	63.6	68.8	11.44
	Full	0.1	88.2	77.3	47.6	93.2	71.8	99.8	79.6	74.4	85.6	85.0	42.3	63.2	69.0	11.43
	Full	0.01	87.6	76.9	46.4	93.0	70.4	99.8	79.0	74.9	88.0	84.0	43.0	63.4	69.6*	11.41
	Full	0	88.0	77.4	48.4	93.3	70.8	99.8	79.6	74.2	87.2	82.0	43.0	64.2	69.1*	11.41
	LoRA	1	88.2	77.0	46.9	92.7	68.8	100.0	78.9	75.4	83.2	79.5	41.3	62.8	66.7	11.41
	LoRA	0.1	88.4	77.7	48.0	93.0	70.6	100.0	79.6	74.7	86.8	78.5	42.0	63.6	67.7	11.42
	LoRA	0.01	88.8	78.2	47.6	92.8	71.0	100.0	79.8	74.1	86.8	81.0	43.3	63.2	68.6	11.43
	LoRA	0	88.0	78.2	47.1	92.3	71.2	100.0	79.5	73.6	87.2	80.5	42.0	64.8	68.6	11.40
0.6B	original	–	72.6	46.8	22.2	54.2	53.6	97.8	57.9	22.2	44.4	55.0	4.0	39.0	35.6	27.52
	Full	1	74.4	47.8	20.5	60.3	53.6	97.8	59.1	60.5	48.4	51.0	4.0	39.8	35.8	26.64
	Full	0.1	75.0	47.2	20.4	60.0	55.4	97.5	59.2	62.2	48.8	52.0	2.3	41.2	36.1	26.50
	Full	0.01	74.6	47.6	20.8	59.0	55.0	97.2	59.0	62.4	50.8*	51.5	3.0	41.2	36.6	26.32
	Full	0	74.0	46.5	22.2	59.2	52.6	97.2	58.6	61.6	54.0*	50.0	2.3	40.2	36.6	26.30
	LoRA	1	72.4	46.0	20.8	57.8	55.0	97.8	58.3	54.9	44.0	53.0	3.7	38.8	34.9	26.19
	LoRA	0.1	71.6	47.5	22.0	58.7	56.0	97.8	58.9	60.2	44.0	51.5	2.3	40.6	34.6	26.37
	LoRA	0.01	72.8	47.0	21.5	57.2	55.2	97.5	58.5	59.0	47.2	52.0	2.3	40.6	35.5	26.22
	LoRA	0	71.4	46.0	20.6	56.5	54.6	97.8	57.8	59.8	48.4	51.0	1.7*	39.4	35.1	26.28
	LoRA, LR 3×10^{-4}	1	69.6	46.4	21.0	57.7	54.8	98.0	57.9	57.9	45.2	53.5	3.0	38.6	35.1	26.75

Table 13: Per-task breakdown of ablation configurations (accuracy in %; lower perplexity is better). Pro: MMLU-Pro; Wino: WinoGrande; B77: Banking77. * Statistically significant shift from the base model ($p < 0.05$).

Model	Method	λ	Drift (development sets)			JevBench text mode		Chat (560 replies)				
			KL start	KL answer	Tree loss	After [k]	At limit	No end	New/res.	Loops	New/res.	Shorter/longer
4B	original	–	0	0	0.763	0	0	69	–	1	–	–
	Full	1	.0011	.0088	0.410	0	0	69	20/20	3	3/1	103/35*
	Full	0.1	.0022	.017	0.417	0	0	62	13/20	1	1/1	105/34*
	Full	0.01	.0033	.029	0.416	1	0	68	16/17	2	2/1	112/30*
	Full	0	.048	.305	0.421	69	44	67	19/21	2	2/1	105/36*
	LoRA	1	.0007	.0045	0.433	0	0	61	15/23	1	1/1	90/43*
	LoRA	0.1	.0020	.014	0.424	0	0	63	16/22	2	2/1	90/45*
	LoRA	0.01	.0057	.029	0.424	2	0	59	12/22	2	2/1	115/24*
	LoRA	0	.222	.114	0.433	9	1	62	16/23	1	1/1	100/41*
0.6B	original	–	0	0	3.158	219	0	3	–	6	–	–
	Full	1	.062	.076	0.838	230	1	9	7/1	12	11/5	85/40*
	Full	0.1	.146	.109	0.833	221	6	14	13/2*	13	12/5	67/61
	Full	0.01	.294	.179	0.832	216	26	13	12/2*	13	13/6	75/61
	Full	0	3.19	1.18	0.822	231	176	21	20/2*	16	14/4*	67/71
	LoRA	1	.050	.052	0.921	223	1	9	7/1	13	11/4	60/58
	LoRA	0.1	.138	.127	0.847	216	1	12	11/2*	11	9/4	61/66
	LoRA	0.01	.261	.185	0.867	213	4	20	19/2*	20	19/5*	84/47*
	LoRA	0	2.17	.753	0.877	231	108	39	38/2*	33	32/5*	59/78
	LoRA, LR 3×10^{-4}	1	.065	.067	0.831	227	1	11	10/2*	12	10/4	83/47*

Table 14: Auxiliary distribution drift and conversational behavior across ablations. KL start and KL answer denote mean KL divergence against base models at the reply prefix and post-bracket positions across 12 validation sets. Text mode counts syntax runaways out of 231 JevBench queries. Chat reports non-terminating (No end) and looping responses on MT-Bench and Dolly-15k. * $p < 0.05$.

Model	Anchors	JevBench		External		General	Behavior		Drift	
		Acc.	ECE	Macro	B77	Avg.	Text	Chat	KL start	KL answer
4B	all	80.1	.070	78.9	75.2	68.8	0	20/20	.0011	.0088
	no context positions	80.1	.089	79.3	75.8 [†]	68.0	3	15/18	.010	.021
	no off-path nodes	78.8	.071	78.6	73.4 [†]	68.4	0	14/17	.0012	.010
	legal mass only ($k = 0$)	80.5	.070	79.3	74.8	69.0	16	26/18	.104	.136
	none ($\lambda = 0$)	79.7	.084	79.6 [†]	74.2 [†]	69.1	69	19/21	.048	.305
0.6B	all	58.0	.118	59.1	60.5	35.8	1	7/1	.062	.076
	no context positions	57.6	.143	58.7	60.8	35.6	3	9/2	.786	.199
	no off-path nodes	57.1	.165	58.8	61.0	35.2	1	12/1*	.066	.077
	legal mass only ($k = 0$)	57.6	.132	59.0	62.0 [†]	37.2	39	14/2*	2.52	.860
	none ($\lambda = 0$)	59.3	.141	58.6	61.6 [†]	36.6	176	20/2*	3.19	1.18

Table 15: Anchor-site ablations on Reason+Long-12k under full-parameter training (accuracy in %; $\lambda = 1$). Ablations omit context positions ($\mathcal{L}_{\text{pos}}$), off-path nodes, or both distribution anchors ($\mathcal{L}_{\text{out}}$, $\mathcal{L}_{\text{pos}}$; equivalent to $k = 0$). [†] Significant shift from all anchors ($p < 0.05$). * Significant conversational shift from base model.

System	Base	Training	Readout	Public	Sealed
Imajev-4B (mo-hit67890, 2026)	Qwen3.5-4B	LoRA in four stages (human data, pseudo-labels, teacher questions, soft labels), weight-averaged	new head + temperature	86.1	37.0
Plumb-4B (crh225, 2026)	JevK5 v0.2	LoRA over five rounds: teacher questions, mined errors, long documents, replayed public data	letter logits + temperature	89.6	38.0
decider-4b v2 (Mapika, 2026)	Qwen3.5-4B-Base	full SFT, then LoRA; about 95 public sets, programmatic and teacher data	letter logits + temperature	83.5	34.7
Jev 1.13.0 (Almeida, 2026)	undisclosed	RLCD (details undisclosed)	–	86.6	36.7
JevK5 v0.2 (allebee, 2026)	Qwen3.5-4B	distilled LoRA: 3,272 teacher questions plus equal public replay	letter logits + temperature	85.3	33.1
Cygnat (blockbrain-ai, 2026)	Gemma-4-12B-it	none	letter logits + temperature	87.9	33.8
Hopper (HopitAI, 2026)	Qwen3.5-4B	LoRA; synthetic families + public data	letter logits + per-type temperature	82.3	34.1
Winnow-12B (Eldan-Ring, 2026)	Gemma-4-12B-it	LoRA; private synthetic and teacher data	answer-token logits	85.7	33.1
reflex 4B (kshetrajna12, 2026)	Qwen3.5-4B	LoRA (the authors now ship the frozen model)	label logits + temperature	79.2	28.2
SemIf (Lee, 2026)	Qwen3.5-4B	none	letter logits	81.0	26.3
open-alternative-jev (IkerMoel, 2026)	Qwen3.5-4B	none	letter logits + temperature	74.0	24.4
LLM2Jev (Yinsongxu, 2026)	Qwen3.5-4B	none	yes/no logits, one prompt per option	76.2 [§]	–
jqv (Octalab, 2026)	Qwen3-32B	none	letter logits + temperature	80.1	28.2
kev 0.6B (Palmer, 2026)	Qwen3-0.6B-Base	LoRA + pointer head; public and programmatic data	new head	66.7	24.0
Decision Fast (FlyMy.AI, 2026)	Qwen3-0.6B-Base	LoRA + pointer head	new head + temperature	63.2	25.6
LLM-as-Jev training-free (ours)	4B, Qwen3.5-4B	none	multi-token suffix	81.4 [‡]	–
LLM-as-Jev 4B, LoRA (ours)	Qwen3.5-4B	LoRA + KL anchors; public data	multi-token suffix	84.0 [‡]	–
LLM-as-Jev Reason+Long-12k (ours)	0.6B, Qwen3-0.6B	full parameters + KL anchors; public data	multi-token suffix	58.0 [‡]	–

Table 16: Community Jev-style models on JevBench v1.4.2.2. [‡] Public-item accuracy from our evaluation; local reruns reproduce official standings within four items. [§] Reported by the project from its own run on the public items.